Data Compression, Gambling and Prediction

Lecture notes on Chapters 5 and 6 of Cover and Thomas,

with a closing session on compression, intelligence and language models

Information theory course
IME-USP

Draft of September 14, 2026

Contents

Notation and standing conventions **2**

1 Codes, trees and the Kraft inequality **3**
1.1 Source codes and their kinds . . . 3
1.2 Code trees and the Kraft inequality . . . 4
1.3 Uniquely decodable codes gain nothing: McMillan's theorem . . . 6

2 The entropy bound, Shannon codes and the price of a wrong model **6**
2.1 The entropy bound . . . 6
2.2 Shannon codes: within one bit . . . 7
2.3 Coding blocks of symbols: the entropy rate . . . 8
2.4 The wrong code: relative entropy as a cost in bits . . . 9

3 Huffman codes **10**
3.1 The algorithm . . . 10
3.2 Three readings of a Huffman code . . . 10
3.3 Optimality . . . 12
3.4 What this buys us . . . 13

4 Coding with the cumulative distribution, competitive optimality, and making randomness **13**
4.1 Shannon–Fano–Elias coding . . . 13
4.2 Competitive optimality of the Shannon code . . . 15

5 Gambling and information: the horse race **17**
5.1 The horse race and the doubling rate . . . 17
5.2 Fair, superfair and subfair odds . . . 18
5.3 The value of side information . . . 19

6 Gambling is compression: the entropy of English **20**
6.1 Sequential betting is a probability assignment . . . 20
6.2 The gambler as a compressor . . . 20
6.3 Models of English . . . 21
6.4 Estimating the entropy of English with people . . . 22

7 Compression, prediction and intelligence: from Shannon to language models 23
7.1 Three names for one number 23
7.2 The shortest description 25
7.3 Compression as a test of intelligence 26
7.4 What the thesis does and does not say 28
7.5 The final activity suggestion: reading a small language model as a compressor . 29

References 30

Notation and standing conventions

Logarithms are base 2 and information is measured in bits; ln denotes the natural logarithm when it appears. Prerequisites from Chapters 2 to 4 of Cover and Thomas [3] are used freely: entropy $H(X)$, relative entropy $D(p\,\|\,q)$ with $D(p\,\|\,q) \geq 0$ and equality iff (if and only if) $p = q$, mutual information $I(X;Y) = H(X) - H(X|Y)$, the chain rule, the entropy rate $H(\mathcal{X}) = \lim \frac{1}{n} H(X_1, \ldots, X_n)$ of a stationary process (we have being considering only i.i.d. sequences), and the weak law of large numbers.

Symbol	Meaning (where defined)
$\mathcal{X},\ x$	source alphabet (finite unless said otherwise) and a source symbol
$\mathcal{D},\ D,\ \mathcal{D}^*$	code alphabet $\{0, \ldots, D-1\}$, its size, and the finite strings over it (Session 1)
$C(x),\ l(x)$	codeword of x and its length; codewords are typeset as `0110` (Session 1)
$L(C) = \sum_x p(x) l(x)$	expected codeword length (Session 1)
L^*	expected length of an optimal (shortest expected length) prefix code (Session 2)
L_n	expected length per source symbol when blocks of n symbols are coded (Session 2)
$H_D(X)$	entropy in base D, $H_D = H/\log D$ (Session 2)
$h(q) = -q \log q - (1-q)\log(1-q)$	binary entropy function - we also use $H(p)$ (Session 2)
$\lceil x \rceil$	smallest integer $\geq x$ (Session 2)
$F(x),\ \overline{F}(x)$	cumulative distribution function and its midpoint version $\overline{F}(x) = F(x-1) + \frac{1}{2}p(x)$ (Session 4)
$\lfloor F \rfloor_l$	the number F truncated to its first l binary digits (Session 4)
T	number of fair coin flips used to generate one sample (Session 4)
$o_i,\ b_i$	odds (o_i-for-1) and fraction of wealth bet on horse i (Session 5)
$S(X) = b(X)o(X),\ S_n$	wealth relative after one race, wealth after n races (Session 5)
$W(b,p) = \mathbb{E} \log S(X)$	doubling rate of the strategy b under the true distribution p (Session 5)
$W^*(p)$	optimal doubling rate $\max_b W(b,p)$ (Session 5)
$b(x_k \mid x^{k-1}),\ b(x^n)$	sequential bets and the probability they assign to a whole sequence (Session 6)
$q(x_k \mid x^{k-1})$	a predictive (language) model, read as a sequential bet (Session 7)

Convention 0.1 *(Standing conventions)*.

(i) $x^n = (x_1, \ldots, x_n)$ denotes a sequence and $x^{k-1} = (x_1, \ldots, x_{k-1})$ its prefix; $C(x^n) = C(x_1) \cdots C(x_n)$ is concatenation.

(ii) Probabilities of source symbols are strictly positive unless stated; a symbol of probability 0 is dropped from the alphabet.

(iii) "Optimal code" means a prefix code of minimum expected length for the given distribution. Session 1 shows that allowing all uniquely decodable codes changes nothing.

(iv) Odds are quoted a-for-1: one unit staked returns a units, including the stake, if the horse wins and nothing otherwise. Fair odds satisfy $\sum_i 1/o_i = 1$.

1 Codes, trees and the Kraft inequality

The first part of the course defined entropy and proved its properties. This session and the next supply the missing link to coding: how to use those properties to design codes that are efficient in terms of bits per symbol. A code assigns bit strings to source symbols, the lengths of the strings are constrained by an inequality that looks like a probability distribution, and the expected length is then bounded below by the entropy. We organize these ideas and results here.

1.1 Source codes and their kinds

Definition 1.1 *(Source code, expected length).* Let X be a random variable with values in a finite set $\mathcal{X}$ and probability mass function p. A *source code* for X over the *code alphabet* $\mathcal{D} = \{0, 1, \ldots, D-1\}$ is a map $C\colon \mathcal{X} \to \mathcal{D}^*$ into the finite strings over $\mathcal{D}$. The string $C(x)$ is the *codeword* of x and $l(x)$ its length. The *expected length* of the code is

$$L(C) = \sum_{x \in \mathcal{X}} p(x)\, l(x) = \mathbb{E}\, l(X). \tag{1}$$

We want to send sequences of symbols, so codewords are concatenated: the *extension* of C is the map $C(x_1 x_2 \cdots x_n) = C(x_1)C(x_2)\cdots C(x_n)$ on finite source strings. A code is useless if the receiver cannot undo this, and there are three grades of usefulness.

Definition 1.2 *(Nonsingular, uniquely decodable, prefix codes).* A code is *nonsingular* when C is injective: distinct symbols have distinct codewords. It is *uniquely decodable* when its extension is injective: distinct source strings, of any lengths, have distinct code strings. It is a *prefix code* (also *instantaneous code*) when no codeword is a prefix of another codeword.

Every prefix code is uniquely decodable, and every uniquely decodable code is nonsingular (Figure 1). A prefix code is uniquely decodable because the receiver can decode on the fly: reading the code string from the left, the moment the bits read so far form a codeword, that codeword is complete, since no longer codeword begins the same way. The code is *self-punctuating*: commas could be inserted without looking ahead. A uniquely decodable code that is not a prefix code may force the receiver to look far ahead before the first symbol is known, as Example 1.4 shows, but the message is still recoverable.

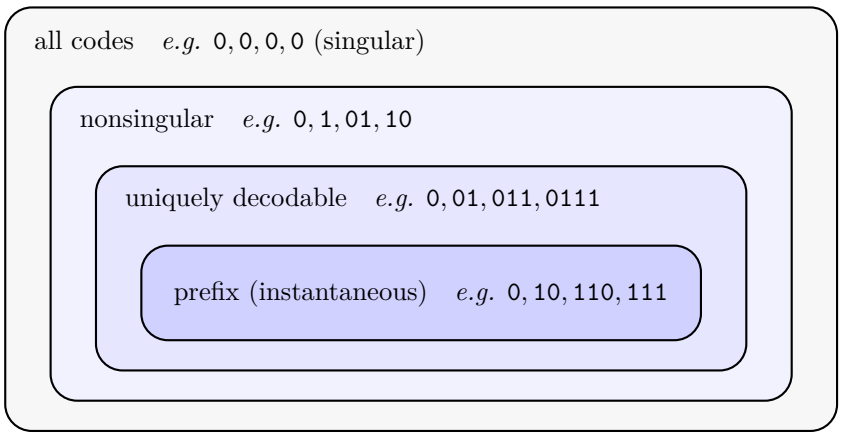


Figure 1: The nested classes of codes for a four-symbol alphabet $\{a, b, c, d\}$, each with an example. The inclusions are strict: each example lies in its own region and not in the smaller one.

Example 1.3 *(Four codes for four symbols).* Let $\mathcal{X} = \{a, b, c, d\}$ and $\mathcal{D} = \{0, 1\}$.

(i) $C(a) = C(b) = C(c) = C(d) = \texttt{0}$ is singular: even a single symbol cannot be recovered.

(ii) $C = (\texttt{0}, \texttt{1}, \texttt{01}, \texttt{10})$ is nonsingular but not uniquely decodable: the code string $\texttt{010}$ is $\texttt{0}\,\texttt{10} = ad$ and also $\texttt{01}\,\texttt{0} = ca$.

(iii) $C = (\texttt{0}, \texttt{01}, \texttt{011}, \texttt{0111})$ is uniquely decodable but not a prefix code: $\texttt{0}$ is a prefix of every other codeword. Example 1.4 shows how to decode it.

(iv) $C = (\texttt{0}, \texttt{10}, \texttt{110}, \texttt{111})$ is a prefix code. The string $\texttt{0110111100110}$ is parsed from the left as $\texttt{0}\,\texttt{110}\,\texttt{111}\,\texttt{10}\,\texttt{0}\,\texttt{110} = acdbac$, each codeword being recognized at its last bit.

Example 1.4 *(Decoding a uniquely decodable code that is not prefix-free).* In the code $(\texttt{0}, \texttt{01}, \texttt{011}, \texttt{0111})$ every codeword starts with $\texttt{0}$ and contains no other $\texttt{0}$. So in any code string the zeros mark the beginnings of codewords: the string $\texttt{0011010111}$ splits at its zeros into $\texttt{0}\,\texttt{011}\,\texttt{01}\,\texttt{0111} = acbd$. The code is uniquely decodable, but the decoder cannot output a upon reading the first $\texttt{0}$; it must wait for the next $\texttt{0}$ (or the end of the message) to learn that the first codeword was $\texttt{0}$ and not $\texttt{01}$. Reversing every codeword gives $(\texttt{0}, \texttt{10}, \texttt{110}, \texttt{1110})$, a prefix code: this is a *suffix code*, and the delay is the price of reading it forwards. Cover and Thomas give a different example, $(\texttt{10}, \texttt{00}, \texttt{11}, \texttt{110})$, whose decoding requires counting the parity of a run of zeros).

Remark 1.5 *(Why prefix codes).* The advantage of a prefix code is decoding without delay and without memory of the future. Theorem 1.9 below shows that there is no disadvantage: whatever codeword lengths a uniquely decodable code can have, a prefix code can have too. From Session 2 on, "code" means "prefix code" unless said otherwise.

1.2 Code trees and the Kraft inequality

A D-ary code is a set of paths in the infinite D-ary tree: from the root, the first code symbol chooses one of D branches, the second symbol one of the D branches below it, and a codeword of length l ends at a node of depth l (Figure 2). The prefix condition says exactly that no codeword node is an ancestor of another codeword node: codewords sit at the leaves of the subtree they span. This picture converts a question about strings into a question about counting nodes.

Theorem 1.6 *(Kraft inequality).* *The codeword lengths $l_1, \dots, l_m$ of any D-ary prefix code satisfy*

$$\sum_{i=1}^{m} D^{-l_i} \le 1. \tag{2}$$

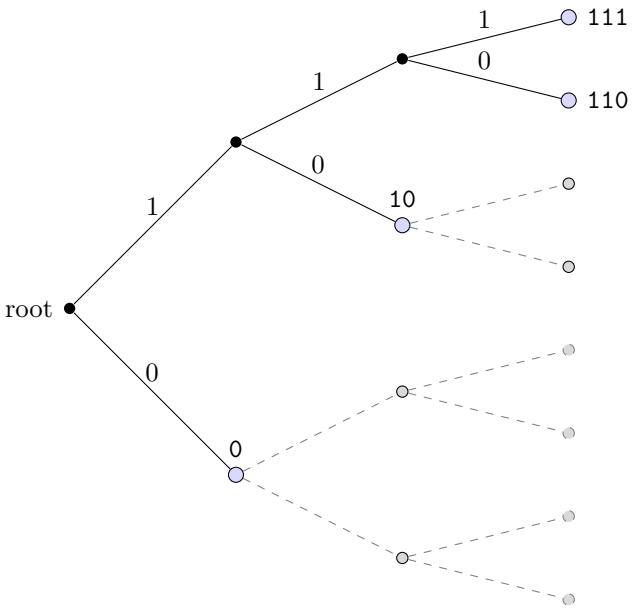
Conversely, for any positive integers $l_1, \dots, l_m$ satisfying (2) there is a D-ary prefix code with exactly these codeword lengths.

Proof. Let $l_{\max}$ be the largest length and consider the $D^{l_{\max}}$ nodes at depth $l_{\max}$ of the D-ary tree. A codeword at depth l_i has $D^{l_{\max}-l_i}$ descendants at depth $l_{\max}$. By the prefix condition no codeword is an ancestor of another, so these descendant sets are pairwise disjoint, and their total size cannot exceed the number of nodes available:

$$\sum_{i=1}^{m} D^{l_{\max}-l_i} \le D^{l_{\max}}.$$

Dividing by $D^{l_{\max}}$ gives (2).

For the converse, order the lengths $l_1 \le l_2 \le \cdots \le l_m$ and assign codewords greedily: take the first (lexicographically smallest) free node at depth l_1, declare it a codeword and delete all its descendants from the tree; then take the first free node at depth l_2, and so on. The step



depth 3: 8 nodes; 0 uses 4, 10 uses 2, 110 and 111 use 1 each

Figure 2: The code tree of the prefix code $0, 10, 110, 111$. A codeword at depth l has 2^{3-l} descendants at depth 3 (grey), and the prefix condition makes these sets disjoint. Counting them proves the Kraft inequality: $4 + 2 + 1 + 1 \le 8$.

for l_k fails only if all D^{l_k} nodes at depth l_k have been deleted, that is, if $\sum_{i<k} D^{l_k - l_i} \ge D^{l_k}$, which after division by D^{l_k} contradicts $\sum_{i<k} D^{-l_i} < \sum_{i \le m} D^{-l_i} \le 1$. So all m codewords are assigned, and by construction none is an ancestor of another. □

The theorem says that a set of codeword lengths is feasible exactly when the numbers D^{-l_i} behave like probabilities that sum to at most one. Session 2 turns this remark into the entropy bound.

The condition is on the lengths only: which strings are used does not matter, and the converse tells us how to build the strings once the lengths are chosen.

Example 1.7 *(Which length lists are feasible).* Binary codes with lengths $(1, 2, 3, 3)$ have Kraft sum $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$, and the code $0, 10, 110, 111$ realizes them with equality: every depth-3 node is used (Figure 2). The lengths $(1, 2, 3, 3, 4)$ have sum $\frac{17}{16} > 1$, so no prefix code, and by Theorem 1.9 no uniquely decodable code, has these lengths. The lengths $(2, 2, 2, 3, 3)$ have sum $\frac{3}{4} + \frac{1}{4} = 1$; the greedy construction gives $00, 01, 10$ at depth 2, then the first free node at depth 3 is 110 and the next 111. For the codes of Example 1.3, the Kraft sums are 2, $\frac{3}{2}$, $\frac{15}{16}$ and 1: the two codes that fail to be uniquely decodable violate the inequality, the other two satisfy it, as Theorem 1.9 requires.

There is a second proof of the Kraft inequality that replaces nodes of a tree by subintervals of $[0, 1)$. It costs nothing extra, it handles infinitely many codewords, and it is the seed of arithmetic coding in Session 4.

Theorem 1.8 *(Extended Kraft inequality).* *The codeword lengths $l_1, l_2, \ldots$ of any countable D-ary prefix code satisfy $\sum_{i \ge 1} D^{-l_i} \le 1$. Conversely, any sequence of positive integers with $\sum_i D^{-l_i} \le 1$ is the length sequence of some D-ary prefix code.*

Proof. Read a codeword $y_1 y_2 \cdots y_l$ over $\mathcal{D}$ as the D-ary fraction $0.y_1 y_2 \cdots y_l = \sum_{j=1}^{l} y_j D^{-j}$ and

associate to it the interval

$$I(y_1 \cdots y_l) = \big[\, 0.y_1 \cdots y_l,\ 0.y_1 \cdots y_l + D^{-l} \,\big), \tag{3}$$

the set of numbers in $[0, 1)$ whose D-ary expansion begins with $y_1 \cdots y_l$. Its length is D^{-l}. A string is a prefix of another exactly when the interval of the first contains the interval of the second, so the prefix condition says that the intervals of distinct codewords are disjoint (Figure 3). Disjoint subintervals of $[0, 1)$ have total length at most 1, which is the inequality. For the converse, order the lengths increasingly and place the intervals one after the other from the left end of $[0, 1)$: the k-th interval starts at $\sum_{i<k} D^{-l_i}$, which is a multiple of D^{-l_k} because all earlier lengths are $\le l_k$, so it is of the form (3) for a unique string of length l_k, and the intervals are disjoint by construction and fit inside $[0, 1)$ by the hypothesis. □

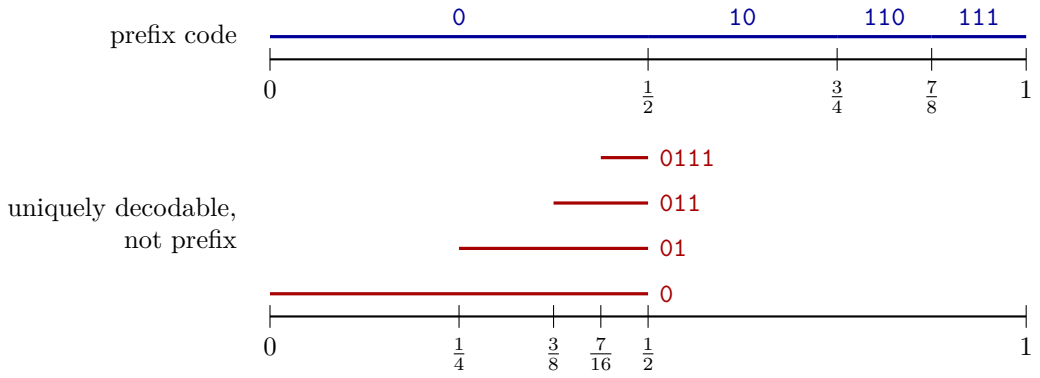


Figure 3: Codewords as subintervals of $[0, 1)$. Top: the prefix code $\mathtt{0}, \mathtt{10}, \mathtt{110}, \mathtt{111}$ partitions the unit interval, and the Kraft sum is the total length, 1. Bottom: the uniquely decodable code $\mathtt{0}, \mathtt{01}, \mathtt{011}, \mathtt{0111}$ has nested intervals, which is another way of seeing that it is not prefix-free. Its total length, counted with multiplicity, is $\frac{15}{16}$.

1.3 Uniquely decodable codes gain nothing: McMillan's theorem

One might hope that dropping the prefix requirement, while keeping unique decodability, allows shorter codewords. It does not as we state below without proof (in this version of the text).

Theorem 1.9 *(McMillan).* *The codeword lengths of any uniquely decodable D-ary code satisfy the Kraft inequality (2). Consequently, for every uniquely decodable code there is a prefix code with the same codeword lengths, hence the same expected length.*

2 The entropy bound, Shannon codes and the price of a wrong model

Session 1 reduced code design to a problem about lengths: minimize $L = \sum_x p(x) l(x)$ over positive integers $l(x)$ with $\sum_x D^{-l(x)} \le 1$. This session solves it up to one bit, and the solution is the entropy.

2.1 The entropy bound

Ignore for a moment that lengths must be integers, and treat the $l(x)$ as real numbers with the constraint $\sum_x D^{-l(x)} = 1$. The Lagrangian $J = \sum_x p(x) l(x) + \lambda \sum_x D^{-l(x)}$ has $\partial J / \partial l(x) =$

$p(x) - \lambda D^{-l(x)} \ln D$, which vanishes when $D^{-l(x)}$ is proportional to $p(x)$; the constraint then forces $D^{-l(x)} = p(x)$, that is

$$l^*(x) = \log_D \frac{1}{p(x)}, \qquad \text{with} \qquad \sum_x p(x) l^*(x) = \sum_x p(x) \log_D \frac{1}{p(x)} = H_D(X). \tag{4}$$

Here $H_D(X) = H(X)/\log D$ is the entropy in base D, in D-ary digits per symbol. The calculus suggests that the ideal length of a symbol is the logarithm of its improbability and that the ideal expected length is the entropy. We prove the bound directly, and the proof exhibits the gap as a relative entropy.

Theorem 2.1 *(The entropy bound).* *The expected length of any D-ary prefix code (or uniquely decodable code) for X satisfies*

$$L \geq H_D(X), \tag{5}$$

with equality if and only if $D^{-l(x)} = p(x)$ for every x.

Proof. Let $c = \sum_x D^{-l(x)}$, which is at most 1 by the Kraft inequality (Theorems 1.6 and 1.9), and let $r(x) = D^{-l(x)}/c$, a probability mass function on $\mathcal{X}$. Then $l(x) = -\log_D r(x) - \log_D c$ and

$$\begin{aligned} L - H_D(X) &= \sum_x p(x) \Big(\log_D \frac{1}{r(x)} - \log_D c \Big) - \sum_x p(x) \log_D \frac{1}{p(x)} \\ &= \sum_x p(x) \log_D \frac{p(x)}{r(x)} + \log_D \frac{1}{c} = \frac{D(p \,\|\, r)}{\log D} + \log_D \frac{1}{c}. \end{aligned} \tag{6}$$

Both terms are nonnegative: the first because relative entropy is nonnegative, the second because $c \leq 1$. Equality requires $r = p$ and $c = 1$, that is $D^{-l(x)} = p(x)$. □

Equation (6) is more informative than the inequality. It says that the excess length of a code over the entropy has two sources: the mismatch $D(p \,\|\, r)$ between the source and the "implied distribution" $r(x) \propto D^{-l(x)}$ of the code, and the slack $\log_D(1/c)$ of an unused part of the Kraft budget. A distribution for which the ideal lengths (4) are integers is called *D-adic* (dyadic for $D = 2$); for a dyadic source the optimal code is exact, $L^* = H(X)$. The four-symbol source $(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$ with the code $\mathtt{0}, \mathtt{10}, \mathtt{110}, \mathtt{111}$ of Session 1 is the standard example: $H = 1.75$ bits and $L = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75$.

2.2 Shannon codes: within one bit

When $\log_D(1/p(x))$ is not an integer, round it up.

Definition 2.2 *(Shannon code).* The *Shannon codeword lengths* for p are $l(x) = \lceil \log_D(1/p(x)) \rceil$, and a *Shannon code* is any prefix code with these lengths.

The lengths are feasible: $\sum_x D^{-\lceil \log_D(1/p(x)) \rceil} \leq \sum_x D^{-\log_D(1/p(x))} = \sum_x p(x) = 1$, so Theorem 1.6 supplies the codewords. Since $\log_D(1/p(x)) \leq l(x) < \log_D(1/p(x)) + 1$, multiplying by $p(x)$ and summing gives $H_D(X) \leq L < H_D(X) + 1$. An optimal code can only do better on the upper side, and Theorem 2.1 controls the lower side.

Theorem 2.3 *(Optimal codes are within one digit of the entropy).* *Let L^* be the expected length of an optimal D-ary prefix code for X. Then*

$$H_D(X) \leq L^* < H_D(X) + 1. \tag{7}$$

Proof. The lower bound is Theorem 2.1. For the upper bound, $L^* \leq L_{\text{Shannon}} < H_D(X) + 1$. □

Example 2.4 *(A five-symbol source, end to end).* Let $p = (0.25, 0.25, 0.2, 0.15, 0.15)$ on $\{1, \dots, 5\}$. The entropy is $H = 2.2855$ bits. The ideal lengths $\log(1/p)$ are $(2, 2, 2.32, 2.74, 2.74)$ and the Shannon lengths $\lceil \log(1/p) \rceil = (2, 2, 3, 3, 3)$, with Kraft sum $\frac{1}{4} + \frac{1}{4} + \frac{1}{8} \cdot 3 = \frac{7}{8} \leq 1$ and expected length $L = 0.5 + 0.5 + 0.6 + 0.45 + 0.45 = 2.5$ bits. The greedy construction of Theorem 1.6 gives the codewords `00`, `01`, `100`, `101`, `110`; the node `111` is left unused, which is the slack $\log_2(8/7) = 0.193$ bits in (6). The decomposition (6) accounts for the whole excess: with $r = (\frac{2}{7}, \frac{2}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7})$, $D(p \,\|\, r) = 0.0219$ and $\log(1/c) = 0.1926$, summing to $L - H = 0.2145$. Session 3 will find the optimal code for this source, of expected length 2.3 bits, still above the entropy but by less. All three numbers obey (7): $2.2855 \leq 2.3 \leq 2.5 < 3.2855$.

2.3 Coding blocks of symbols: the entropy rate

The overhead of at most one digit per symbol in (7) comes from rounding, and rounding can be shared. Code n source symbols at a time, regarding $X^n = (X_1, \dots, X_n)$ as a single symbol from the alphabet $\mathcal{X}^n$, and let L_n be the expected length *per source symbol*, $L_n = \frac{1}{n}\mathbb{E}\, l(X_1, \dots, X_n)$. Applying Theorem 2.3 to the supersymbol,

$$H(X_1, \dots, X_n) \leq \mathbb{E}\, l(X^n) < H(X_1, \dots, X_n) + 1, \tag{8}$$

for the optimal block code (we take $D = 2$ from here on). For i.i.d. symbols $H(X^n) = nH(X)$, and dividing by n,

$$H(X) \leq L_n^* < H(X) + \frac{1}{n}. \tag{9}$$

The rounding overhead is now $1/n$ per symbol, and the entropy is approached as closely as desired by coding long enough blocks.

This is Shannon's source coding theorem in its simplest form, and it is the operational meaning of entropy:

Entropy is the minimum expected number of bits per symbol needed to describe the source.

Example 2.5 *(A biased coin coded in blocks).* Let X_i be i.i.d. Bernoulli with $\mathbb{P}(X_i = 1) = 0.1$, so $H(X) = h(0.1) = 0.4690$ bits. Coding one symbol at a time, the only prefix codes have lengths $(1, 1)$ and $L_1^* = 1$ bit per symbol, more than twice the entropy. For $n = 2$ the four blocks have probabilities $(0.81, 0.09, 0.09, 0.01)$; the optimal code (Session 3) has lengths $(1, 2, 3, 3)$, expected length $0.81 + 0.18 + 0.27 + 0.03 = 1.29$ bits per block and $L_2^* = 0.645$ bits per symbol. For $n = 3$ the optimal code over the eight blocks gives $L_3^* = 0.5327$. The sequence $1,\ 0.645,\ 0.533, \dots$ descends towards 0.469, and each term respects (9): $L_n^* < h(0.1) + 1/n$. The bound is loose, but the trend is the theorem.

Nothing in (8) used independence. For any source, stationary or not, the optimal block code satisfies

$$\frac{H(X_1, \dots, X_n)}{n} \leq L_n^* < \frac{H(X_1, \dots, X_n)}{n} + \frac{1}{n}, \tag{10}$$

and for a stationary process the left side converges (Chapter 4) to the entropy rate $H(\mathcal{X}) = \lim_n \frac{1}{n} H(X_1, \dots, X_n)$.

Theorem 2.6 *(Entropy rate is the compression limit).* *For a stationary source, the minimum expected description length per symbol of an optimal block code satisfies $L_n^* \to H(\mathcal{X})$ as $n \to \infty$.*

This is the second justification of the entropy rate, after its definition in Chapter 4: it is the number of bits per symbol needed to describe the process. Session 6 estimates this number for English, and Session 7 explains why the training loss of a language model is an estimate of the same quantity.

2.4 The wrong code: relative entropy as a cost in bits

Suppose the code is designed for a distribution q but the symbols are drawn from p, because q is our best model of an unknown source. The Shannon lengths $l(x) = \lceil \log(1/q(x)) \rceil$ are still a valid prefix code, since they satisfy the Kraft inequality for q.

What do they cost under p?

Theorem 2.7 *(Wrong code).* *The expected length under p of the Shannon code designed for q satisfies*

$$H(p) + D(p \,\|\, q) \;\le\; \mathbb{E}_p\, l(X) \;<\; H(p) + D(p \,\|\, q) + 1. \tag{11}$$

Proof. $\mathbb{E}_p l(X) = \sum_x p(x) \lceil \log \frac{1}{q(x)} \rceil$ lies between $\sum_x p(x) \log \frac{1}{q(x)}$ and that quantity plus one, and

$$\sum_x p(x) \log \frac{1}{q(x)} = \sum_x p(x) \log \frac{p(x)}{q(x)} + \sum_x p(x) \log \frac{1}{p(x)} = D(p \,\|\, q) + H(p). \tag{12}$$

□

As we saw before, the quantity (12), $\mathbb{E}_p[\log(1/q(X))]$, is called the *cross-entropy* of q relative to p.

The theorem gives relative entropy its most concrete meaning in the course: $D(p \,\|\, q)$ is the number of extra bits per symbol paid for modelling the source as q when it is really p. Believing the wrong model costs exactly the divergence between the truth and the belief.

Example 2.8 *(Two wrong models).* Let the source be $p = (\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$, $H(p) = 1.75$ bits. A coder who assumes the four symbols equally likely, $q = (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$, uses two bits per symbol; the theorem predicts $H(p) + D(p \,\|\, q) = 1.75 + 0.25 = 2$, and here the lower bound is attained because q is dyadic. A coder who assumes $q = (0.7, 0.1, 0.1, 0.1)$ uses Shannon lengths $(1, 4, 4, 4)$ and pays $\mathbb{E}_p l = 0.5 \cdot 1 + 0.5 \cdot 4 = 2.5$ bits, against $H(p) + D(p \,\|\, q) = 1.75 + 0.1683 = 1.9183$: within the promised one bit, with the rounding taking most of the difference. The lesson is not the exact number but its shape: the penalty is a divergence, and a model that assigns small probability to something that happens often ($q(2) = 0.1$ against $p(2) = 0.25$) pays for it in bits.

Remark 2.9 *(Looking ahead to Session 7).* Write (12) for a whole sequence: a model $q(x^n)$ of sequences, used to code $X^n \sim p$, costs $\mathbb{E}_p \log(1/q(X^n))$ bits up to one, and the ideal, unattainable coder that knows p costs $H(X^n)$. The average $\frac{1}{n}\mathbb{E}_p \log(1/q(X^n))$ is what machine learning calls the cross-entropy loss of the model q on data from p, and minimizing it over a family of models is minimizing the length of the code that q defines. We shall come back to this identity when the models are language models.

Historical note 2.10. Theorems 2.1, 2.3 and the block coding argument are Shannon's, from Shannon [20], where the codes with lengths $\lceil \log(1/p) \rceil$ appear together with the construction Shannon attributed to Fano. The reading of $D(p \,\|\, q)$ as the redundancy of a code designed for the wrong distribution is implicit in the same paper and is standard.

3 Huffman codes

Session 2 showed how to get within one bit of the entropy. This session finds the exact optimum: a greedy algorithm, due to Huffman, that builds the code tree from the leaves up by repeatedly merging the two least probable symbols.

3.1 The algorithm

Definition 3.1 *(Huffman's algorithm, binary case).* Given probabilities $p_1, \dots, p_m$, repeat until a single symbol remains: take the two symbols of smallest probability, replace them by one new symbol whose probability is their sum, and record that the two are the children of the new one. The recorded merges form a binary tree with the original symbols as leaves; label the two edges below each internal node `0` and `1`, and read the codeword of each leaf along the path from the root.

Each merge reduces the number of symbols by one, so after $m-1$ merges the tree is complete. Ties may be broken arbitrarily; different tie-breaks give different codes of the same expected length (Example 3.4). The expected length can be read off the merges directly: each merge of probabilities a and b adds one bit to every leaf below it, so

$$L = \sum_{\text{merges}} (a+b) = \text{sum of the probabilities of the internal nodes.} \tag{13}$$

Example 3.2 *(A six-symbol source, binary).* Let $p = (0.3, 0.2, 0.15, 0.15, 0.1, 0.1)$ on $x_1, \dots, x_6$, with $H = 2.4710$ bits. The merges are $0.1 + 0.1 = 0.2$ (symbols x_5, x_6), then $0.15 + 0.15 = 0.3$ (x_3, x_4), then $0.2 + 0.2 = 0.4$ (x_2 with the node $\{x_5, x_6\}$), then $0.3 + 0.3 = 0.6$ (x_1 with $\{x_3, x_4\}$), and finally $0.6 + 0.4 = 1$. Figure 4 shows the tree and the codewords `10, 00, 110, 111, 010, 011`, with lengths $(2, 2, 3, 3, 3, 3)$. By (13), $L = 0.2 + 0.3 + 0.4 + 0.6 + 1.0 = 2.5$ bits, and directly $0.6 + 0.4 + 0.45 + 0.45 + 0.3 + 0.3 = 2.5$. The Kraft sum is $\frac{1}{4} + \frac{1}{4} + 4 \cdot \frac{1}{8} = 1$: a Huffman code always uses the whole budget, because every internal node has two children. Compare $L = 2.5$ with $H = 2.471$: the optimal code is 0.029 bits above the entropy, and the theorem of this session says no prefix code does better.

For a D-ary code alphabet, merge the D least probable symbols at each step. Each merge reduces the number of symbols by $D - 1$, so the process ends with exactly one symbol only if $m \equiv 1 \pmod{D-1}$; otherwise add *dummy symbols* of probability 0 until it does. Without them the first merge would leave an internal node with fewer than D children at the deepest level, wasting part of the Kraft budget where it hurts most.

Example 3.3 *(The same source, ternary).* With $D = 3$ and $m = 6$ we need $m \equiv 1 \pmod 2$, so one dummy symbol of probability 0 is added. Merging three at a time: $0 + 0.1 + 0.1 = 0.2$, then $0.15 + 0.15 + 0.2 = 0.5$ (either 0.2 may be chosen), then $0.3 + 0.2 + 0.5 = 1$. One tie-break gives lengths $(1, 2, 2, 2, 2, 2)$ in trits and expected length $L = 0.3 \cdot 1 + 0.7 \cdot 2 = 1.7$ trits, the other gives $(1, 1, 2, 2, 3, 3)$ and again 1.7. The entropy in trits is $H / \log_2 3 = 1.559$. The bound $H_D \le L^* < H_D + 1$ reads $1.559 \le 1.7 < 2.559$.

3.2 Three readings of a Huffman code

Twenty questions. A sequence of yes/no questions that identifies an object is the same thing as a binary prefix code for the set of objects: the answers, written as bits, form the codeword, the questions correspond to the internal nodes of the tree, and the expected number of questions is the expected codeword length. An optimal questioning strategy for a known

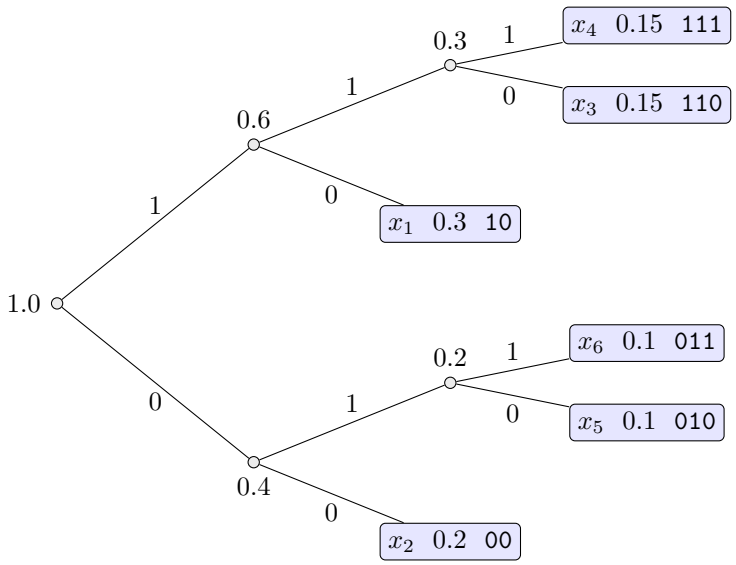


Figure 4: The Huffman tree of Example 3.2. Internal nodes carry the merged probabilities; the expected length 2.5 is their sum. Leaves are labelled with symbol, probability and codeword. Swapping the labels `0` and `1` on any node gives another optimal code.

distribution on the objects is therefore a Huffman code, and by Theorem 2.3 the expected number of questions $\mathbb{E}Q$ satisfies $H(X) \le \mathbb{E}Q < H(X) + 1$. In Example 3.2 the first question is "is $X \in \{x_1, x_3, x_4\}$?", the two subtrees of the root. Note that the questions need not be asked adaptively: the second bit of the codeword is `0` exactly for x_1 and x_2, so the second question can be "is $X \in \{x_1, x_2\}$?" whatever the first answer was.

Weighted lengths and file merging. The algorithm never uses that the p_i sum to one: for any nonnegative weights w_i it minimizes $\sum_i w_i l_i$ over prefix codes. Read the weights as sizes of sorted files and a merge of two files as a pass that costs their combined size: by (13) the total cost of merging all files into one by pairwise merges is $\sum_i w_i l_i$, and Huffman's order minimizes it. For files of sizes $(5, 3, 2, 1)$ the merges $1 + 2 = 3$, $3 + 3 = 6$, $5 + 6 = 11$ cost $3 + 6 + 11 = 20$ units, against $1 + 5 = 6$, $6 + 2 = 8$, $8 + 3 = 11$ costing 25 for a poor order.

Slice codes. Huffman's questions are arbitrary subsets. If the symbols are ordered $p_1 \ge \cdots \ge p_m$ and only questions "is $X > a$?" are allowed, take the Huffman *lengths* $l_1 \le \cdots \le l_m$ (they are ordered this way by Lemma 3.5) and assign codewords by the greedy construction of Theorem 1.6, first free node at each depth in symbol order. The result has the same lengths, hence is also optimal, and its codewords are in alphabetical order, so every bit splits the ordered alphabet into a lower and an upper slice. For Example 2.4, the Huffman lengths $(2, 2, 2, 3, 3)$ give the slice code `00, 01, 10, 110, 111` of length 2.3.

Example 3.4 *(Huffman versus Shannon on a single symbol; optimal lengths are not unique).* Let $p = (\frac{1}{3}, \frac{1}{3}, \frac{1}{4}, \frac{1}{12})$. The first merge is forced, $\frac{1}{12} + \frac{1}{4} = \frac{1}{3}$, and then three symbols of probability $\frac{1}{3}$ remain. Merging the two original ones gives lengths $(2, 2, 2, 2)$; merging one original with the new node gives $(1, 2, 3, 3)$. Both have $L = 2$: $\frac{1}{3} \cdot 1 + \frac{1}{3} \cdot 2 + \frac{1}{4} \cdot 3 + \frac{1}{12} \cdot 3 = 2$. So the multiset of optimal lengths is not unique, only the optimal value is. In the second code the symbol of probability $\frac{1}{4}$ gets length 3, while its Shannon length is $\lceil \log 4 \rceil = 2$: a Huffman code can be longer than the Shannon code on an individual symbol, although never on average. The reverse happens too: for $p = (0.9999, 0.0001)$ the Shannon lengths are $(1, 14)$ and the Huffman lengths $(1, 1)$. Optimality is a statement about the average,

and Session 4 asks what can be said symbol by symbol.

3.3 Optimality

Throughout, $p_1 \geq p_2 \geq \cdots \geq p_m$ and a code is optimal if it minimizes $\sum_i p_i l_i$ among binary prefix codes. There are many optimal codes (exchange two codewords of the same length, or complement a bit at some node), and the proof works with a representative of a convenient shape.

Lemma 3.5 *(A canonical optimal code exists).* *For every distribution with $p_1 \geq \cdots \geq p_m$ there is an optimal prefix code such that*

(i) lengths are ordered inversely to probabilities: $p_j > p_k$ implies $l_j \leq l_k$;
(ii) the two longest codewords have the same length;
(iii) the two longest codewords differ only in their last bit and belong to the two least probable symbols x_{m-1}, x_m.

Proof. Start from any optimal code C. (i) *Swap.* If $p_j > p_k$ and $l_j > l_k$, exchanging the codewords of x_j and x_k changes the expected length by $p_j l_k + p_k l_j - p_j l_j - p_k l_k = (p_j - p_k)(l_k - l_j) < 0$, contradicting optimality. So (i) holds for every optimal code. (ii) *Trim.* If the longest codeword were strictly longer than all others, deleting its last bit would keep the code prefix-free (no other codeword can be a prefix of the shortened word, since none has its length or more) and shorten the expected length, again a contradiction. So at least two codewords have the maximal length. The same argument shows that every codeword of maximal length has a *sibling*, a codeword differing from it only in the last bit: a maximal-length codeword without sibling could be trimmed. (iii) *Rearrange.* By (i) the least probable symbols have maximal length. Among the maximal-length codewords, all of which come in sibling pairs, exchange codewords (which does not change any length, hence not the expected length) so that x_{m-1} and x_m occupy one sibling pair. □

A code with these three properties is called *canonical*. Its tree has the two least probable symbols as siblings at the bottom, which is exactly the shape of the first Huffman merge.

Theorem 3.6 *(Huffman codes are optimal).* *If C^* is a Huffman code for p and C' is any uniquely decodable code for p, then $L(C^*) \leq L(C')$.*

Proof. By Theorem 1.9 it is enough to compare with prefix codes. We prove by induction on m that the Huffman code for $p = (p_1, \ldots, p_m)$, $p_1 \geq \cdots \geq p_m$, is optimal. For $m = 2$ the only prefix codes have lengths $(1, 1)$ and every code is optimal.

Let $m \geq 3$ and define the *Huffman reduction* $p' = (p_1, \ldots, p_{m-2}, p_{m-1} + p_m)$ on $m - 1$ symbols, which is the distribution after the first merge. Two constructions relate codes for p and codes for p'.

Expansion. From a prefix code for p' with lengths $l'_1, \ldots, l'_{m-1}$, build a code for p by keeping the codewords of $x_1, \ldots, x_{m-2}$ and giving x_{m-1}, x_m the codeword of the merged symbol extended by `0` and by `1`. The result is a prefix code, and its expected length is

$$L(p) = \sum_{i \leq m-2} p_i l'_i + (p_{m-1} + p_m)(l'_{m-1} + 1) = L'(p') + p_{m-1} + p_m. \tag{14}$$

Condensation. From a canonical optimal code for p (Lemma 3.5), build a code for p' by deleting the last bit of the sibling pair of x_{m-1}, x_m and assigning the common prefix to the merged symbol. The result is a prefix code for p' with expected length

$$L(p') = \sum_{i \leq m-2} p_i l_i + (p_{m-1} + p_m)(l_m - 1) = L^*(p) - p_{m-1} - p_m, \tag{15}$$

where $L^*(p)$ is the optimal expected length for p.

Now apply expansion to an optimal code for p', of length $L^*(p')$, obtaining a code for p of length $L^*(p') + p_{m-1} + p_m$; and apply condensation to a canonical optimal code for p, obtaining a code for p' of length $L^*(p) - p_{m-1} - p_m$. Each constructed code is at least as long as the optimum for its distribution:

$$L^*(p') + p_{m-1} + p_m \;\geq\; L^*(p), \qquad L^*(p) - p_{m-1} - p_m \;\geq\; L^*(p').$$

These two inequalities together force equality in both: $L^*(p) = L^*(p') + p_{m-1} + p_m$. In words, expanding an optimal code for the reduction p' yields an optimal code for p. By the induction hypothesis the Huffman code for p' is optimal, and the Huffman code for p is exactly its expansion (the first merge of p produces p', and the remaining merges are the Huffman construction for p'). Hence the Huffman code for p is optimal. □

The argument is a template for proving that a greedy algorithm is optimal: show that some optimal solution has the shape of the greedy first step (the lemma), and that the problem after the step is the same problem on a smaller instance with the cost shifted by a constant (the two equations). The D-ary case follows the same lines, with dummy symbols making the tree full.

Historical note 3.7. Huffman [8] found the algorithm as a graduate student in Fano's information theory class at MIT, as an alternative to writing a final examination; it is one of the most widely deployed algorithms in computing, inside JPEG, MP3, ZIP and many other formats, usually as the final stage of a longer pipeline. The suboptimal top-down construction that splits the ordered symbols into two halves of nearly equal probability is Fano's; Shannon's own construction, with lengths $\lceil \log(1/p) \rceil$, is the code of Session 2.

3.4 What this buys us

For a single symbol drawn from a known distribution, the compression problem is solved: Huffman's algorithm produces an optimal prefix code, its expected length lies in $[H, H+1)$, and no uniquely decodable code does better. The remaining inefficiency, up to one bit per symbol, is a rounding effect that block coding removes in principle, but Huffman coding on blocks of length n requires a tree with $|\mathcal{X}|^n$ leaves. Session 4 shows how to code long sequences at $\log(1/p(x^n))$ bits without building that tree.

4 Coding with the cumulative distribution, competitive optimality, and making randomness

Three shorter topics complete Chapter 5. The first replaces the search for an optimal code by a formula: read the codeword of x off the cumulative distribution function. It costs one extra bit, but it needs no tree, it works for any ordering of the symbols, and, applied to a sequence one symbol at a time, it becomes arithmetic coding, the method behind most modern compressors and the bridge to Sessions 6 and 7. The second topic asks what "optimal" means for a single outcome rather than on average. The third reverses the direction of compression: how many fair coin flips does it take to produce a sample from a given distribution?

4.1 Shannon–Fano–Elias coding

Let $\mathcal{X} = \{1, \ldots, m\}$ with all $p(x) > 0$, and let $F(x) = \sum_{a \leq x} p(a)$ be the cumulative distribution function, a staircase with a step of height $p(x)$ at each x. The midpoint of the step at x,

$$\overline{F}(x) = \sum_{a<x} p(a) + \tfrac{1}{2}p(x) = F(x-1) + \tfrac{1}{2}p(x), \tag{16}$$

determines x: the midpoints of distinct steps are distinct numbers, so the real number $\overline{F}(x)$ is already a (very long) description of x. The question is how many binary digits of $\overline{F}(x)$ are enough.

Theorem 4.1 *(Shannon–Fano–Elias code).* *Let $l(x) = \lceil \log(1/p(x)) \rceil + 1$ and let $C(x)$ be the first $l(x)$ binary digits of $\overline{F}(x)$, so that $C(x)$ read as a binary fraction is $\lfloor \overline{F}(x) \rfloor_{l(x)}$. Then C is a prefix code and*

$$L(C) = \sum_x p(x) l(x) < H(X) + 2. \tag{17}$$

Proof. Truncation to l digits loses less than 2^{-l}, and $2^{-l(x)} \le \frac{1}{2} p(x)$ by the choice of $l(x)$. So $\lfloor \overline{F}(x) \rfloor_{l(x)}$ lies in $(\overline{F}(x) - \frac{1}{2}p(x), \overline{F}(x)] = (F(x-1), \overline{F}(x)]$, inside the step of x: the truncated number still identifies x. For the prefix property use the interval picture of Session 1: the codeword $C(x)$ corresponds to the interval $[\lfloor \overline{F}(x) \rfloor_{l(x)},\ \lfloor \overline{F}(x) \rfloor_{l(x)} + 2^{-l(x)})$ of length at most $\frac{1}{2}p(x)$, whose left end is in the lower half of the step of x; hence the interval lies entirely inside $[F(x-1), F(x))$. Steps of different symbols are disjoint, so the codeword intervals are disjoint, and by Theorem 1.8 the code is prefix-free. Finally $l(x) < \log(1/p(x)) + 2$, and averaging gives (17). □

Example 4.2 *(A four-symbol source).* For $p = (0.4, 0.3, 0.2, 0.1)$ the steps are $[0, 0.4), [0.4, 0.7), [0.7, 0.9), [0.9, 1)$ with midpoints $\overline{F} = (0.2, 0.55, 0.8, 0.95)$, and $l = \lceil \log(1/p) \rceil + 1 = (3, 3, 4, 5)$. In binary, $0.2 = 0.0011\,0011\ldots$, $0.55 = 0.1000\,1100\ldots$, $0.8 = 0.1100\,1100\ldots$, $0.95 = 0.1111\,0011\ldots$, and truncating gives the codewords `001`, `100`, `1100`, `11110` (Figure 5). No codeword is a prefix of another. The expected length is $1.2 + 0.9 + 0.8 + 0.5 = 3.4$ bits against $H = 1.846$: comfortably below the guaranteed $H + 2 = 3.846$, and comfortably above the Huffman length 1.9 for this source. Two bits per symbol is a real price, and it is paid to gain a formula and, in the next paragraph, sequences.

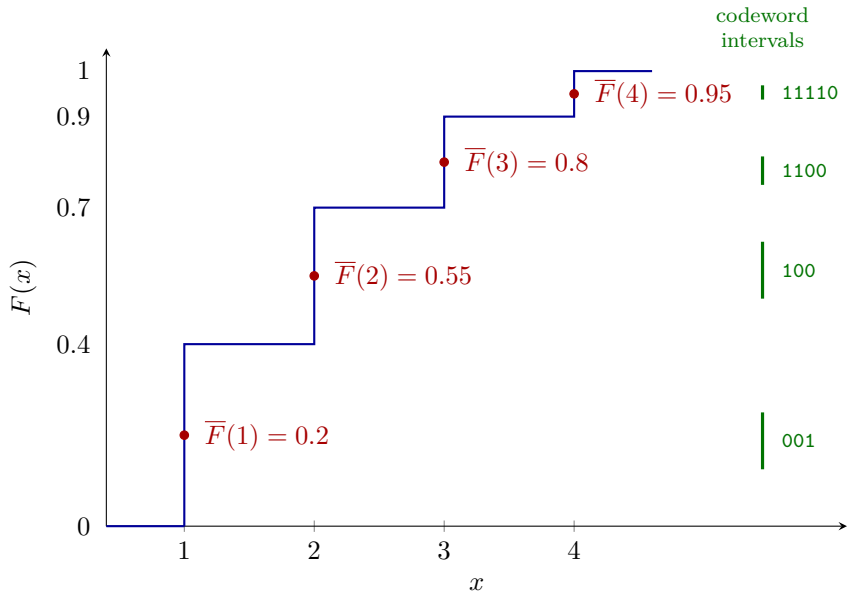


Figure 5: Shannon–Fano–Elias coding for $p = (0.4, 0.3, 0.2, 0.1)$. The staircase is F, the dots are the midpoints $\overline{F}(x)$, and the brackets on the right mark the intervals $[\lfloor \overline{F} \rfloor_l, \lfloor \overline{F} \rfloor_l + 2^{-l})$ of the four codewords, each inside its own step. The codewords are the binary digits of the brackets' lower ends.

From one symbol to a sequence: arithmetic coding. Nothing in Theorem 4.1 needs the alphabet to be small. Apply it to the alphabet $\mathcal{X}^n$ of sequences ordered lexicographically: the codeword of x^n is $\overline{F}(x^n)$ truncated to $\lceil \log(1/p(x^n)) \rceil + 1$ digits, and the expected length is below $H(X^n) + 2$, so the overhead per symbol is at most $2/n$ and the entropy rate is approached, as in Theorem 2.6. The apparent obstacle is that $F(x^n)$ is a sum over all sequences preceding x^n, exponentially many. The remedy is to compute it sequentially. The step of x^n is the interval

$$[\, F(x^n - 1),\, F(x^n)\,) \quad \text{of length } p(x^n), \tag{18}$$

where $x^n - 1$ denotes the lexicographic predecessor, and this interval is obtained by refining the interval of x^{n-1}: the step of x^{n-1}, of length $p(x^{n-1})$, is split into $|\mathcal{X}|$ sub-steps of lengths $p(x^{n-1})\, p(x_n \mid x^{n-1})$, one for each possible next symbol, in alphabetical order, and the step of x^n is the sub-step of the symbol that actually occurred. After n refinements the interval has length $p(x^n)$ and any point inside it, written to $\lceil \log(1/p(x^n)) \rceil + 1$ binary digits, is a codeword that the decoder can invert by repeating the same refinements and watching which sub-step contains the codeword. Each refinement needs only the conditional probabilities $p(x_k \mid x^{k-1})$, so any model that predicts the next symbol from the past, an i.i.d. model, a Markov chain, or the predictive models of Session 7, plugs in directly. What we have described is *arithmetic coding* with exact arithmetic (Pasco and Rissanen, 1976)

Example 4.3 *(Arithmetic coding of a short binary string).* Let X_i be i.i.d. with $\mathbb{P}(X_i = 1) = 0.3$, so a `0` takes the lower 70% of the current interval and a `1` the upper 30%. For $x^8 = \texttt{00100000}$ the intervals are, step by step,

$$\begin{aligned}[0,1) &\to [0,0.7) \to [0,0.49) \to [0.343,0.49) \to [0.343,0.4459) \\ &\to [0.343,0.41503) \to [0.343,0.393421) \to [0.343,0.378295) \to [0.343,0.367706),\end{aligned}$$

the third step taking the upper 30% because $x_3 = 1$ (Figure 6). The final length is $0.7^7 \cdot 0.3 = 0.024706 = p(x^8)$, so $\lceil \log(1/p(x^8)) \rceil + 1 = \lceil 5.339 \rceil + 1 = 7$ digits suffice; the midpoint 0.3554 truncated to 7 digits is $\texttt{0101101} = 0.351562$, which lies in the final interval, and $0.351562 + 2^{-7} = 0.359375$ still does, so the codeword interval is inside the step. Seven bits code eight symbols. That is a modest gain, but the expected cost per symbol of this scheme tends to $h(0.3) = 0.881$ bits for long strings, against one bit per symbol for the raw string, and against the 1 bit per symbol that any code for one binary symbol at a time must pay.

4.2 Competitive optimality of the Shannon code

Huffman codes minimize the expected length. A code designed by someone else may still beat them on a particular symbol, since it is always possible to give a rare symbol a short codeword. Is there a sense in which a good code wins symbol by symbol, most of the time? For Shannon lengths $l(x) = \lceil \log(1/p(x)) \rceil$ the answer is yes, and the proof is another use of the Kraft inequality.

Theorem 4.4 *(No code beats the Shannon code by much, often).* *Let $l(x) = \lceil \log(1/p(x)) \rceil$ and let $l'(x)$ be the lengths of any other uniquely decodable code. Then for every integer $c \geq 1$,*

$$\mathbb{P}\big(l(X) \geq l'(X) + c\big) \leq 2^{-(c-1)}. \tag{19}$$

Proof. If $l(x) \geq l'(x) + c$ then $\log(1/p(x)) > l(x) - 1 \geq l'(x) + c - 1$, that is $p(x) < 2^{-l'(x)-(c-1)}$. Summing $p(x)$ over such x,

$$\mathbb{P}\big(l(X) \geq l'(X) + c\big) \leq \sum_x 2^{-l'(x)}\, 2^{-(c-1)} \leq 2^{-(c-1)}$$

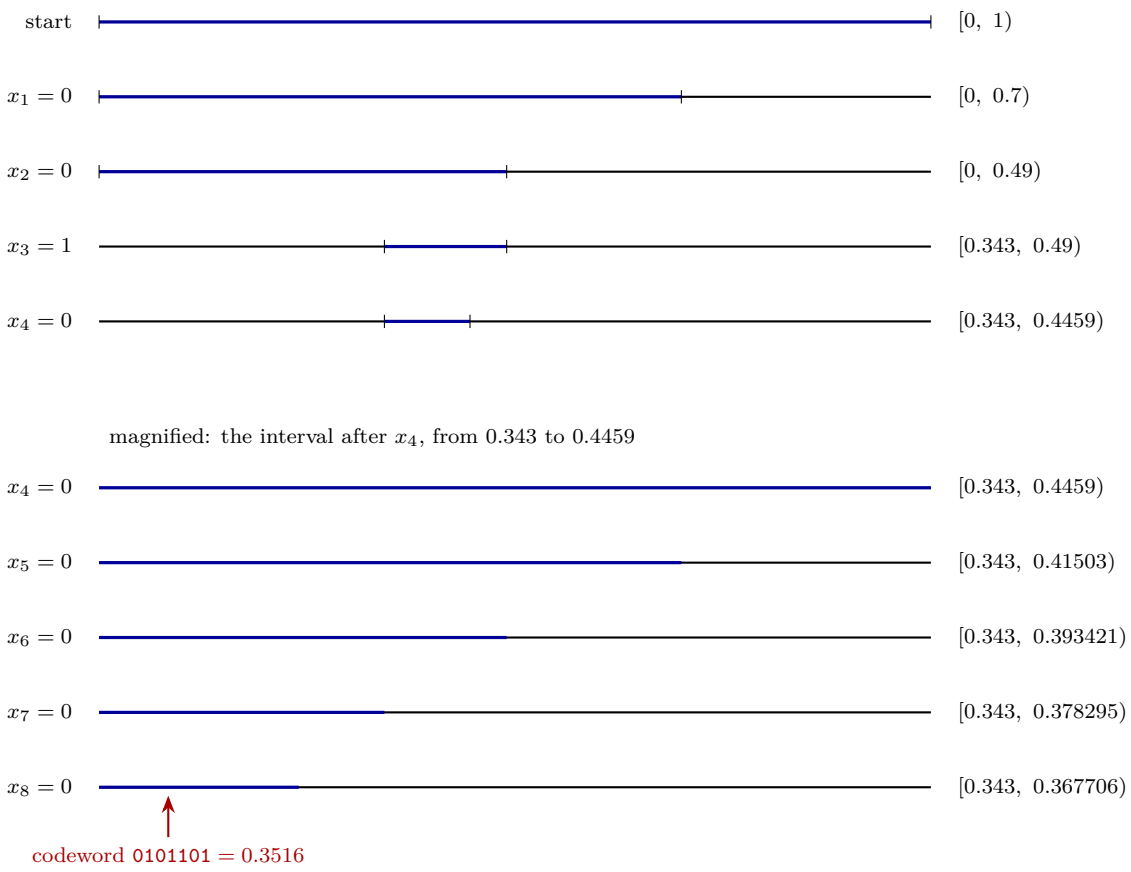


Figure 6: The nested intervals of Example 4.3. Each symbol shrinks the interval by its probability, 0.7 for a `0` and 0.3 for a `1`, and the final interval has length $p(x^8)$. Any binary fraction inside it, with enough digits for its own interval to fit, is a codeword; the decoder finds x^8 by reproducing the refinements.

by the Kraft inequality for l'. □

So the probability that a rival code is 5 or more bits shorter than the Shannon code on the symbol that occurs is below $1/16$, whatever the rival. A sharper statement holds for dyadic sources, where the Shannon lengths are the exact ideal lengths $\log(1/p)$.

Theorem 4.5 *(Competitive optimality for dyadic sources).* *If p is dyadic and $l(x) = \log(1/p(x))$, then for any other uniquely decodable code with lengths l',*

$$\mathbb{P}\big(l(X) < l'(X)\big) \;\geq\; \mathbb{P}\big(l(X) > l'(X)\big), \tag{20}$$

with equality if and only if $l' = l$.

Proof. The function $\operatorname{sgn}(t)$, equal to $1, 0, -1$ for t positive, zero, negative, satisfies $\operatorname{sgn}(t) \leq 2^t - 1$ at every integer t: for $t \geq 1$, $1 \leq 2^t - 1$; for $t = 0$ both sides vanish; for $t \leq -1$, $-1 < 2^t - 1 < 0$. Then

$$\begin{aligned}\mathbb{P}(l' < l) - \mathbb{P}(l' > l) &= \mathbb{E}\,\operatorname{sgn}\big(l(X) - l'(X)\big) \leq \mathbb{E}\big[2^{l(X)-l'(X)} - 1\big] \\ &= \sum_x 2^{-l(x)}\, 2^{l(x)-l'(x)} - 1 = \sum_x 2^{-l'(x)} - 1 \leq 0,\end{aligned}$$

using $p(x) = 2^{-l(x)}$ in the third step and the Kraft inequality in the last. Equality requires $\operatorname{sgn}(t) = 2^t - 1$ at every $t = l(x) - l'(x)$, which holds only for $t \in \{0, 1\}$, and $\sum_x 2^{-l'(x)} = 1$; the two together force $l'(x) = l(x)$ for all x. □

5 Gambling and information: the horse race

A gambler who bets on a horse race must distribute money over outcomes, and a compressor who codes a source must distribute codeword lengths over outcomes. Both are choosing a probability distribution to act on, and both are judged, in the long run, by how close their distribution is to the truth. This session develops the gambler's side of the analogy; the next one shows that the two activities are the same.

5.1 The horse race and the doubling rate

Definition 5.1 *(Horse race, odds, betting strategy).* A *horse race* has m horses; horse i wins with probability p_i, independently from race to race, and pays *odds* o_i-for-1: a unit bet on horse i returns o_i units if it wins and nothing otherwise. A *betting strategy* is a vector $b = (b_1, \dots, b_m)$ with $b_i \geq 0$ and $\sum_i b_i = 1$, the fraction of current wealth placed on each horse. If horse X wins, the gambler's wealth is multiplied by the *wealth relative*

$$S(X) = b(X)\, o(X). \tag{21}$$

The gambler reinvests everything each time, so after n independent races the wealth, starting from 1, is the product $S_n = \prod_{k=1}^{n} S(X_k)$. Products of many random factors are governed by the average of their logarithms.

Definition 5.2 *(Doubling rate).* The *doubling rate* of the strategy b against the win distribution p is

$$W(b, p) = \mathbb{E} \log S(X) = \sum_{i=1}^{m} p_i \log(b_i o_i), \tag{22}$$

and the *optimal doubling rate* is $W^*(p) = \max_b W(b, p)$.

Theorem 5.3 *(Wealth grows at the doubling rate).* *For i.i.d. races, $\frac{1}{n} \log S_n \to W(b, p)$ in probability. In words, $S_n = 2^{nW(b,p)}$ to first order in the exponent: the gambler doubles the wealth W times per race.*

Proof. $\frac{1}{n} \log S_n = \frac{1}{n} \sum_{k=1}^{n} \log S(X_k)$ is the average of i.i.d. random variables with mean $W(b, p)$, and the weak law of large numbers applies (if some $b_i = 0$ with $p_i > 0$ the mean is $-\infty$ and the gambler is ruined with probability one within finitely many races). □

So the gambler should maximize the doubling rate, not the expected wealth after one race. The two criteria disagree sharply: betting everything on the horse with the largest $p_i o_i$ maximizes the expected one-race return and leads to ruin with probability one as soon as that horse loses. The right question is which b maximizes (22), and the answer does not depend on the odds at all.

Theorem 5.4 *(Proportional betting is log-optimal).* *For any odds o,*

$$W(b, p) = \sum_i p_i \log o_i - H(p) - D(p \,\|\, b), \tag{23}$$

so the optimal doubling rate is $W^*(p) = \sum_i p_i \log o_i - H(p)$, attained exactly by the proportional *(or Kelly) strategy $b^* = p$.*

Proof. Write $\log(b_i o_i) = \log o_i + \log p_i + \log(b_i / p_i)$ and take expectations under p: $\sum p_i \log o_i$, then $-H(p)$, then $-D(p \,\|\, b)$. Since $D(p \,\|\, b) \geq 0$ with equality iff $b = p$, the maximum is at $b = p$. □

The identity (23) has the same shape as the gap formula (6) of Session 2: the loss from a suboptimal strategy is the relative entropy between the truth and the strategy, exactly as the excess length of a code was the relative entropy between the source and the code's implied distribution. Betting in proportion to one's beliefs is the gambler's version of assigning length $\log(1/p)$.

Example 5.5 *(A three-horse race).* Three horses win with probabilities $p = (0.6, 0.25, 0.15)$ and pay odds $o = (2, 3, 6)$. The odds are fair, $\sum_i 1/o_i = \frac{1}{2} + \frac{1}{3} + \frac{1}{6} = 1$, and the bookmaker's implied distribution is $r = (1/o_i) = (\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$, which underrates the favourite. The Kelly gambler bets $b = p$ and has

$$W^* = 0.6 \log 2 + 0.25 \log 3 + 0.15 \log 6 - H(0.6, 0.25, 0.15) = 1.3840 - 1.3527 = 0.0313$$

bits per race, a doubling of wealth every $1/0.0313 \approx 32$ races. The gambler who bets the bookmaker's own odds, $b = r$, has $b_i o_i = 1$ for every horse and wealth constant forever, $W = 0$; by (23) this is $W^* - D(p \,\|\, r)$, so $W^* = D(p \,\|\, r)$ here. An aggressive gambler who bets $b = (0.8, 0.1, 0.1)$ on the favourite has $W = 0.6 \log 1.6 + 0.25 \log 0.3 + 0.15 \log 0.6 = -0.1379$ bits per race, and loses a factor of two every seven races despite backing the most likely horse most heavily; $D(p \,\|\, b) = 0.1692$ accounts for the difference from W^*. Figure 7 shows one simulation of 1000 races, common to the three strategies: the Kelly gambler ends at $\log_2 S_{1000} = 32.1$ (about 4.7×10^9 times the initial wealth), the bookmaker-follower at 0, the aggressive gambler at -135.3, and the paths hug the lines nW as Theorem 5.3 predicts.

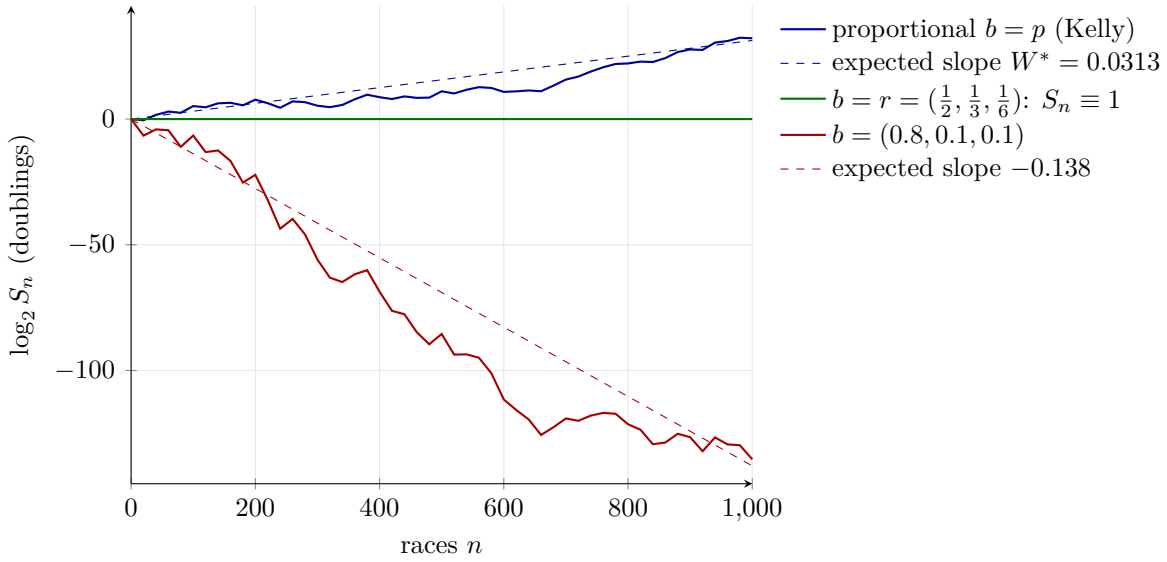


Figure 7: Logarithm of wealth over 1000 simulated races of Example 5.5, the same race outcomes for all three strategies. Each path fluctuates around the straight line of slope $W(b, p)$; the sign of the slope, not the luck of any single race, decides who gets rich.

5.2 Fair, superfair and subfair odds

When $\sum_i 1/o_i = 1$ the odds are *fair* with respect to the distribution $r_i = 1/o_i$, the bookmaker's implied probabilities, and (23) becomes

$$W(b, p) = D(p \,\|\, r) - D(p \,\|\, b). \tag{24}$$

The gambler makes money exactly when his estimate of the race is closer to the truth than the bookmaker's. Betting $b = r$ is the same as keeping the money in cash, since then every horse

returns exactly the stake, so the option of holding cash adds nothing under fair odds. With uniform fair odds $o_i = m$ for all i, (24) gives the cleanest statement of the whole chapter.

Theorem 5.6 *(Conservation law).* *For uniform fair odds $o_i = m$,*

$$W^*(p) + H(p) = \log m. \tag{25}$$

Proof. $W^*(p) = \sum_i p_i \log m - H(p)$ by Theorem 5.4. □

Every bit of entropy the race lacks is a doubling of wealth the gambler gains: predictable races are profitable, and the maximally unpredictable race, the uniform one, yields nothing. The same constant $\log m$ appears in Session 6 as the length of the raw description of a race outcome, and the conservation law becomes "bits saved by the compressor equal doublings won by the gambler".

Remark 5.7. When $\sum_i 1/o_i < 1$ the odds are *superfair*: betting $b_i = c/o_i$ with $c = 1/\sum_j (1/o_j) > 1$ returns c whatever happens, a risk-free profit (a *Dutch book*), though not the largest doubling rate, which is still attained by $b = p$. When $\sum_i 1/o_i > 1$ the odds are *subfair*, the realistic case with a track take, and it may be optimal to keep part of the wealth in cash; proportional betting is then no longer optimal and the solution has a water-filling form (Cover and Thomas, Problem 6.2).

5.3 The value of side information

Suppose the gambler observes, before each race, a random variable Y correlated with the winner X (a tip, the weather, past form), and may choose the bets as a function of y: $b(x \mid y) \geq 0$, $\sum_x b(x \mid y) = 1$. The doubling rates with and without the information are

$$W^*(X \mid Y) = \max_{b(\cdot\mid\cdot)} \sum_{x,y} p(x,y) \log\bigl(b(x \mid y) o(x)\bigr), \qquad W^*(X) = \max_{b} \sum_{x} p(x) \log\bigl(b(x) o(x)\bigr). \tag{26}$$

Theorem 5.8 *(Side information is worth its mutual information).* $W^*(X \mid Y) - W^*(X) = I(X;Y)$.

Proof. For each fixed y, the inner problem in $W^*(X \mid Y)$ is a horse race with win distribution $p(x \mid y)$, so by Theorem 5.4 the optimal bets are $b^*(x \mid y) = p(x \mid y)$ and

$$W^*(X \mid Y) = \sum_{x,y} p(x,y) \log\bigl(p(x \mid y) o(x)\bigr) = \sum_x p(x) \log o(x) - H(X \mid Y).$$

Subtracting $W^*(X) = \sum_x p(x) \log o(x) - H(X)$ gives $H(X) - H(X \mid Y) = I(X;Y)$. □

This is the first place in the course where mutual information "gets a price": it is the increase in the exponential growth rate of wealth, in doublings per race, that the information buys. Independent information is worth nothing, and no information is worth more than $H(X)$, which would make the race certain.

Example 5.9 *(A tip that is right nine times in ten).* Two horses, even odds $o = (2, 2)$, each winning with probability $\frac{1}{2}$. Without information $W^* = 1 - H(\frac{1}{2}, \frac{1}{2}) = 0$: nothing to be gained. A tip Y names the winner with probability 0.9 and the loser with probability 0.1. Then $I(X;Y) = H(X) - H(X \mid Y) = 1 - h(0.1) = 0.5310$ bits, and the gambler who bets $b(x \mid y) = p(x \mid y)$, that is 0.9 on the tipped horse and 0.1 on the other, has doubling rate 0.5310, doubling the wealth roughly every two races.
Betting everything on the tip would be ruinous within about ten races.

Historical note 5.10. The horse race analysis, the identification of the optimal growth rate with $\log m - H$ and the equation $\Delta W = I$ are due to Kelly [13], a colleague of Shannon at Bell Laboratories who set out to give the channel capacity of Shannon's theory an interpretation in terms of money; proportional betting is called the Kelly criterion after him. Cover and Thomas record that log-optimal investment goes back, through Latané, to Daniel Bernoulli's treatment of the St. Petersburg paradox.

6 Gambling is compression: the entropy of English

Session 5 ended with an analogy. This session makes it an identity: a sequential gambler is a probability assignment on sequences, a probability assignment on sequences is an arithmetic code (Session 4), and the bits the code saves equal the doublings the gambler wins. The identity is then used in the other direction, to estimate the entropy rate of English by letting people gamble on text.

6.1 Sequential betting is a probability assignment

Consider a binary sequence $X_1, X_2, \ldots, X_n$ and a gambler who, before each symbol, bets a fraction $b(x_k \mid x^{k-1})$ of the current wealth on each possible value of X_k, at even odds (2-for-1), with $b(0 \mid x^{k-1}) + b(1 \mid x^{k-1}) = 1$. The bets may depend on everything seen so far. After n symbols the wealth, starting from 1, is

$$S_n = \prod_{k=1}^{n} 2\, b(x_k \mid x^{k-1}) = 2^n\, b(x^n), \qquad b(x^n) \coloneqq \prod_{k=1}^{n} b(x_k \mid x^{k-1}). \tag{27}$$

The function $b(x^n)$ is nonnegative and sums to one over the 2^n sequences (multiply out the products and use $\sum_{x_k} b(x_k \mid x^{k-1}) = 1$ at each step from the last symbol backwards). So a sequential betting scheme *is* a probability distribution on sequences, and conversely any distribution $b(x^n)$ on sequences is a sequential scheme through its conditionals $b(x_k \mid x^{k-1}) = b(x^k)/b(x^{k-1})$. Betting on the whole sequence at once, at 2^n-for-1, is the same as betting sequentially. The gambler's bets are his estimate of the distribution of the data, and $S_n/2^n$ is the probability his model assigned to what happened.

6.2 The gambler as a compressor

Give the distribution $b(x^n)$ to the arithmetic coder of Session 4 in place of the true distribution. The codeword of x^n is a binary fraction inside the step of x^n in the cumulative distribution of b, with

$$l(x^n) = \lceil \log \tfrac{1}{b(x^n)} \rceil + 1 = \lceil\, n - \log S_n(x^n) \,\rceil + 1 \;\le\; n - \log S_n(x^n) + 2 \tag{28}$$

bits, and the code is prefix-free by Theorem 4.1. The decoder needs the same distribution b: an "identical twin" of the gambler, who places the same bets on every sequence and can therefore reproduce the interval refinements (Figure 8). The twin need not be a person; it is enough that the betting rule be deterministic and available to both sides, which is what "model" means for a compressor.

Theorem 6.1 *(Bits saved equal doublings won).* *Any deterministic sequential betting scheme b at even odds defines a prefix code for binary sequences of length n whose length on x^n is at most $n - \log S_n(x^n) + 2$: the gambler who multiplies his wealth by S_n on a sequence saves at least $\log S_n - 2$ bits on it, relative to the raw n bits. If the data are drawn from p and the gambler bets proportionally, $b = p$, the expected length is at most $H(X_1, \ldots, X_n) + 2$.*

Proof. The first statement is (28). For the second, $b(x^n) = p(x^n)$ gives

$$\mathbb{E}l(X^n) \le \mathbb{E}\log\frac{1}{p(X^n)} + 2 = H(X^n) + 2. \qquad \square$$

The theorem is the conservation law of Session 5 (Theorem 5.6) written for a whole sequence and for an arbitrary strategy: $n = \log(\text{raw length})$ plays the role of $\log m$, and the identity $\log S_n = n - \log(1/b(x^n))$ splits the raw n bits into the bits the model needs and the bits it saves. Nothing requires the alphabet to be binary; for m symbols at m-for-1 odds, replace n by $n \log m$.

Example 6.2 *(The eight-bit string of Session 4, as a bet).* The i.i.d. model with $\mathbb{P}(1) = 0.3$ is the gambler who bets 0.7 on `0` and 0.3 on `1` at every step. On $x^8 = \texttt{00100000}$ his wealth is $S_8 = 2^8 \cdot 0.7^7 \cdot 0.3 = 6.325$, so $\log S_8 = 2.66$ doublings, and (28) gives $l = \lceil 8 - 2.66 \rceil + 1 = 7$ bits, exactly the codeword `0101101` found in Example 4.3. The gambler who bets $\frac{1}{2}$ on each symbol has $S_8 = 1$ and codes nothing away; a gambler who had bet 0.9 on `0` would have $S_8 = 2^8 \cdot 0.9^7 \cdot 0.1 = 12.25$ on this string, that is $\log S_8 = 3.61$ doublings and a $\lceil 8 - 3.61 \rceil + 1 = 6$-bit codeword, but pays for it on strings with more ones. Same string, three models, three code lengths: compression is a property of the pair (data, model), and the model's wealth is the exact bookkeeping.

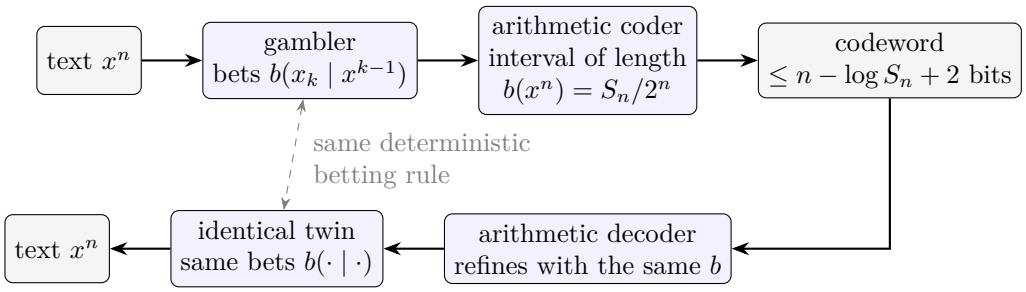


Figure 8: A gambler compresses: the bets define a probability assignment $b(x^n) = S_n/2^n$, which the arithmetic coder turns into a codeword of about $n - \log S_n$ bits. The decoder must own an identical twin of the gambler, that is, the same deterministic betting rule.

6.3 Models of English

The natural source to try this on is language. Take English text over an alphabet of 27 symbols, the letters and the space, ignoring case and punctuation. Whether English is a stationary ergodic process is doubtful; we ask nonetheless for its entropy rate, understood as the number of bits per letter that the best predictor could achieve, and approach it through a sequence of increasingly informed models, each of which yields, by Theorem 2.7, an upper bound.

The zero-order model treats the letters as independent and equally likely: $\log 27 = 4.76$ bits per letter. The first-order model keeps independence but uses the letter frequencies of English (E about 13%, Q and Z about 0.1%): about 4.03 bits per letter. Second-, third- and fourth-order Markov models condition on one, two and three preceding letters; the fourth-order model gives about 2.8 bits per letter. Shannon illustrated these models by sampling text from them, and the samples are a lesson in themselves: the zero-order sample is noise, the second-order sample contains pronounceable fragments ("ON IE ANTSOUTINYS ARE T INCTORE. . . ", from Shannon's 1948 paper), the third-order sample contains English-looking words ("DEMONSTURES OF THE REPTAGIN"), and second-order *word* models produce runs of plausible phrases. The higher the order, the better the compression and the more the samples look like English; the two are the same fact. The obstacle to continuing is data: a fourth-order letter model has $27^4 \approx 531{,}000$

contexts to estimate, and a k-th order word model has astronomically more. Session 7 is about what happens when this obstacle is removed by models with a different architecture and a great deal more text.

6.4 Estimating the entropy of English with people

The best available model of English is an English speaker, and two experiments turn a person into a predictor whose performance bounds the entropy rate from above.

Shannon's guessing game. A subject reads a passage and guesses the next letter, guessing again after each wrong guess, until right; the experimenter records the number of guesses G_k needed at each position. If the subject's guessing were a deterministic function of the preceding text (a strong idealization), the sequence $G_1, G_2, \dots$ would determine the text: a twin subject, given the guess numbers, would reproduce the guesses in the same order and take the G_k-th one. So $H(G_1, \dots, G_n) = H(X_1, \dots, X_n)$, and the entropy rate of the guess sequence is that of the text. The guess sequence is much more concentrated than the text (most guesses are right the first time), and its entropy is bounded above by that of its empirical histogram, since treating the G_k as independent can only increase entropy. Shannon's 1951 experiment obtained in this way about 1.3 bits per letter for English with long contexts.

Example 6.3 *(A guess histogram and its entropy).* Suppose a subject's guesses are right at the first attempt 70% of the time, at the second 15%, and at the third to sixth 7%, 4%, 2%, 2% (hypothetical figures, chosen for illustration). The entropy of this histogram is $H(0.70, 0.15, 0.07, 0.04, 0.02, 0.02) = 1.45$ bits, and if the guessing rule is deterministic this is an upper bound on the entropy rate of the text being read: about 1.45 bits per letter, against 4.76 for the uniform model. The gap between 4.76 and this number is the redundancy of English that the subject's knowledge of the language captures.

The gambling estimate. Guessing throws away information: a subject who thinks the next letter is E with probability 0.6 and A with probability 0.3 can only say "E". Let the subject bet instead. At 27-for-1 on each letter, betting $b(x_k \mid x^{k-1})$, the wealth after n letters is $S_n = 27^n b(x^n)$ by (27) with 27 in place of 2, and

$$\frac{1}{n}\mathbb{E}\log S_n = \log 27 + \frac{1}{n}\mathbb{E}\log b(X^n) = \log 27 - \frac{1}{n}H(X^n) - \frac{1}{n}D(p(x^n)\,\|\,b(x^n)) \;\le\; \log 27 - \frac{1}{n}H(X^n). \tag{29}$$

For a stationary source the right side tends to $\log 27 - H(\mathcal{X})$, so

$$\hat{H} = \log 27 - \frac{1}{n}\log S_n \tag{30}$$

is an estimate of the entropy rate that is, in expectation, an upper bound, with equality exactly when the subject bets the true conditional probabilities. Better gamblers give tighter bounds; a gambler cannot, on average, make the text look more predictable than it is. Cover and King [2] ran this experiment with twelve subjects on 75 letters of English prose and obtained $\hat{H} = 1.34$ bits per letter.

Example 6.4 *(What 1.34 bits per letter means in money).* Solving (30) for the wealth, $\log S_{75} = 75\,(\log 27 - 1.34) = 75 \times 3.415 = 256.1$, so $S_{75} = 2^{256.1} \approx 1.3 \times 10^{77}$. A subject who bets on 75 letters of English at fair 27-for-1 odds, with the skill of Cover and King's subjects, multiplies one dollar by roughly the number of atoms in the observable universe. The same number, read as compression, says that the 75 letters, which occupy $75 \log 27 = 357$ bits raw, can be coded by the subject's twin in about $75 \times 1.34 = 100$ bits.

Historical note 6.5. The stochastic models of English and the sampled texts are in Shannon [20]; the guessing game and the estimate of about 1.3 bits per letter are in Shannon [21]; the gambling estimate is Cover and King [2]. The connection between gambling and coding through sequential probability assignment is due to Kelly (Session 5), and the compressor built from a gambler is, in modern terms, arithmetic coding with an adaptive model, the architecture of the compressors that Session 7 compares.

7 Compression, prediction and intelligence: from Shannon to language models

Sessions 2, 5 and 6 proved three theorems that look different and are the same. A code for a source cannot beat the entropy, and using a wrong distribution costs the relative entropy (Theorems 2.1 and 2.7). A gambler cannot grow faster than $\log m - H$, and betting a wrong distribution costs the relative entropy (Theorems 5.4 and 5.6). A gambler and a compressor are the same object, scored by the same number (Theorem 6.1). This closing session takes those theorems where the book does not: to the systems now called *language models*, which are trained by minimising exactly the score of Theorem 6.1, and to a question that has been asked seriously since the 1960s and can now be tested on a laptop: is the ability to compress a body of text a measure of how much one understands it? The session ends with a measurement protocol for students who want to find out for themselves.

7.1 Three names for one number

Everything in this session is about *sequential probability assignments*: rules that, having seen x^{t-1}, put a probability $q(x_t \mid x^{t-1})$ on the next symbol, for $t = 1, \dots, n$. Multiplying them gives a probability $q(x^n) = \prod_t q(x_t \mid x^{t-1})$ on whole sequences, and every probability on sequences arises this way (Section 6.1). A Markov chain estimated from counts is such a rule; so is a human reader guessing letters; so is a neural network whose last layer is a softmax over the alphabet. Machine learning calls the rule a *model* and scores it by its *cross-entropy loss*.

Definition 7.1 *(Cross-entropy loss, perplexity).* The *log-loss* of the model q on the sequence x^n is

$$L_q(x^n) = \sum_{t=1}^{n} \log \frac{1}{q(x_t \mid x^{t-1})} = \log \frac{1}{q(x^n)} \quad \text{bits}, \tag{31}$$

its *cross-entropy loss* is the per-symbol average $\ell_q(x^n) = L_q(x^n)/n$ bits per symbol, and its *perplexity* is $2^{\ell_q(x^n)}$.

The perplexity is the number of equally likely symbols that would produce the same loss: a model that assigns probability $1/m$ to every symbol has perplexity m, and a model with perplexity 3 is, per symbol, as uncertain as a fair three-sided die. Figure 9 states the whole point of Sessions 5 and 6 in one picture: the number (31) has three readings.

Theorem 7.2 *(Training is coding).* *Let q be any sequential probability assignment on $\mathcal{X}^n$, $|\mathcal{X}| = m$.*

(i) The arithmetic code driven by q (Section 6.2) is a prefix code of length $l_q(x^n) \le L_q(x^n) + 2$ for every x^n; conversely, every prefix code C on $\mathcal{X}^n$ defines a (sub)probability assignment $q_C(x^n) = 2^{-l(x^n)}$ (Theorem 1.6) with $L_{q_C} = l$ exactly.

(ii) The gambler who bets $q(x_t \mid x^{t-1})$ at m-for-1 odds ends with $S_n = m^n\, q(x^n)$, so $L_q(x^n) = n \log m - \log S_n$.

(iii) If $X^n \sim p$, then $\mathbb{E}L_q(X^n) = H(X^n) + D(p \,\|\, q)$, hence $\mathbb{E}\ell_q(X^n) \ge H(X^n)/n$ with equality iff $q = p$; for a stationary source the lower bound tends to the entropy rate $H(\mathcal{X})$.

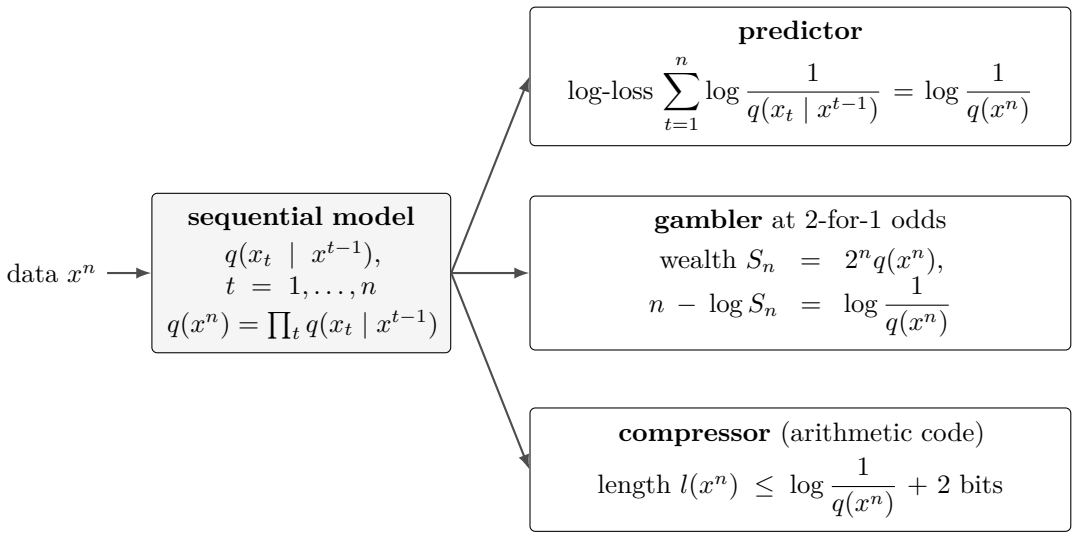


Figure 9: One sequential model, three readings. The predictor's log-loss, the gambler's shortfall $n - \log S_n$ at fair odds, and the length of the arithmetic code built from the model are the same number, up to the two bits of Theorem 6.1.

Proof. Part (i) is Theorem 6.1 with $b = q$ together with the Kraft inequality; part (ii) is (27) with m in place of 2; part (iii) is Theorem 2.7 applied to the sequence as one symbol, followed by Theorem 2.6. □

The theorem has a consequence that deserves to be stated slowly. Modern language models are trained by gradient descent on the average of (31) over a corpus, usually written in nats with ln in place of log and called the cross-entropy or negative log-likelihood. By part (i), that average is the length in bits, per symbol, of the compressed corpus under the arithmetic coder built from the model; by part (ii), it is the shortfall of the model's wealth as a gambler at fair odds; by part (iii), its expected value exceeds the entropy rate of the source by the relative entropy between truth and model. *Training a language model is building a compressor, and the training loss is the compressed size.* The reported loss of a model on a held-out text is a literal claim about a file: multiply it by the number of symbols and you have, within two bits, the number of bits the model's arithmetic coder would write.

Remark 7.3 *(Units).* Three conversions are needed to read the literature. A loss of λ nats per symbol is $\lambda / \ln 2$ bits per symbol and a perplexity of $e^{\lambda} = 2^{\lambda / \ln 2}$. A loss per *token* (the sub-word units into which language models split text) becomes a loss per character or per byte on dividing by the average number of characters or bytes per token, a number that depends on the tokeniser and the text. Compressors report bits per byte of the original file. Comparisons across models, compressors and Shannon's 1.3 bits per letter are meaningful only after everything is converted to bits per character of the *same* text; Section 7.5 makes this a rule.

The twin problem. Theorem 7.2(i) produces a codeword, but a codeword is useless unless the decoder can reproduce, symbol by symbol, the very probabilities the encoder used (Figure 8). For a table of counts this is a formality; for a network with 10^9 parameters it is the whole difficulty, and there are exactly two ways out. The *two-part* way transmits a description of the

model first, say its parameters, then the arithmetic code: total length $L(\text{model}) + L_q(x^n) + 2$. The *online* (or *prequential*) way fixes a starting model, a random seed and an update rule that both parties know, and lets encoder and decoder train identical copies on the symbols already coded, so that nothing about the model is transmitted and the model learns from the file itself. Adaptive arithmetic coders [24], the context-mixing compressors that hold the records of Section 7.3, and NNCP [1], which trains a Transformer while compressing and retrains its twin while decompressing, are online; a language model trained once and then evaluated is two-part, with the model's description length usually left off the bill. The two accountings agree on very long files and disagree violently on short ones: a model that has memorised the file compresses it to nothing under the online rule's cousin, "evaluate on the training data", and to more than its own size under the two-part rule. Section 7.2 gives this bookkeeping its proper name.

7.2 The shortest description

Shannon's theory scores a code against a source *distribution*. Three related ideas from the 1960s and 1970s remove the distribution and score a description against the *data alone*, and they supply the vocabulary in which the compression-and-intelligence thesis is stated.

Kolmogorov complexity. Fix a universal computer U. The *Kolmogorov complexity* $K(x)$ of a finite string x is the length in bits of the shortest program that makes U print x and halt [14, 22]. Changing U changes K by at most an additive constant, so K is well defined up to $O(1)$, and four facts about it are all this session needs [16, Chapters 2–4]. First, $K(x) \le |x| + O(1)$: one can always print the string literally. Second, most strings are incompressible: fewer than 2^{n-k} strings of length n have $K(x) < n - k$, because there are fewer than 2^{n-k} programs that short, which is the counting argument of Theorem 1.6 once more. Third, K is *not computable*: no algorithm returns $K(x)$ for all x. Fourth, and this is what makes the notion useful in practice, every compressor gives a computable *upper bound*: if C compresses x to $l(x)$ bits and the decompressor is a program of d bits, then $K(x) \le l(x) + d + O(1)$. A compression benchmark is a contest of upper bounds on the Kolmogorov complexity of a fixed file, and the Hutter Prize's rule of counting the decompressor's size (Section 7.3) makes the sum $l(x) + d$ literally a program that prints the file.

Solomonoff's universal predictor. Solomonoff asked what one should predict after seeing x^{t-1} if one knows nothing about the source except that it is computable, and answered with a sequential probability assignment: weight every program by $2^{-\text{length}}$ and let $M(x)$ be the total weight of programs whose output begins with x. This is a mixture over all computable sources with prior weights $2^{-K(\mu)}$, roughly, and it has the property that matters.

Theorem 7.4 *(Dominance and the total regret of M). For every computable source μ on infinite sequences there is a constant $c_\mu = K(\mu) + O(1)$ such that $M(x^n) \ge 2^{-c_\mu}\mu(x^n)$ for all n and x^n. Consequently*

$$L_M(x^n) - L_\mu(x^n) = \log \frac{\mu(x^n)}{M(x^n)} \le c_\mu \qquad \text{for all } n \text{ and all } x^n. \tag{32}$$

The proof is one line once M is set up properly, since the program for μ is one of the programs in the mixture [16, Section 4.5]; the set-up (semimeasures, enumerability) is where the care goes and we skip it. The statement is worth comparing with Theorem 2.7. There, coding with the wrong distribution costs $D(p \,\|\, q)$ *per symbol*, forever. Here, coding with M instead of the unknown truth costs at most $K(\mu)$ bits *in total*, however long the sequence: the price of ignorance is the description length of what one was ignorant of, paid once. Divided by n, the regret goes to zero, so M's conditional predictions converge to the truth for every

computable source. The price is that M, like K, is not computable; everything that follows can be read as the search for computable stand-ins.

Minimum description length. Rissanen [19] turned the two-part code into a principle of statistics: among competing models for a data set, prefer the one minimising $L(\text{model}) + L(\text{data} \mid \text{model})$, both measured in bits, the first by a code for the model's parameters, the second by the model's arithmetic code for the data. A more complex model shortens the second term and lengthens the first; the minimum is a formal Occam's razor, and it is exactly the two-part accounting of the twin problem. Grünwald [5] is the modern reference. In the language of Session 2, MDL chooses the code, and therefore the gambler, whose total bill, including the cost of saying which gambler it is, is smallest.

7.3 Compression as a test of intelligence

The idea that predicting text well requires knowing what the text is about is older than computers that could do either. Shannon [21] measured 1.3 bits per letter for human readers precisely because readers use grammar, meaning and knowledge of the world to guess the next letter; Cover and King [2] had to give their subjects the same knowledge to get 1.34. The reverse implication, that a machine reaching those numbers must have acquired something of that knowledge, was proposed as a test in Mahoney [18], who argued that a compressor reaching about one bit per character on English text would have to model the language as well as a person does and that the compressed size is an objective, repeatable score where the Turing test is neither. Hutter [10] put the argument on the footing of Section 7.2: Solomonoff's M is the optimal predictor for a general agent, and an agent that can compress its observations well has, in that precise sense, understood them. Legg and Hutter [15] then defined an agent's intelligence as its expected performance across all computable environments weighted by 2^{-K}; compression enters as the induction half of the definition.

The Hutter Prize and `enwik9`. To make the thesis testable, Hutter established a prize in 2006 for compressing `enwik8`, the first 10^8 bytes of an English Wikipedia dump, extended in 2020 to `enwik9`, the first 10^9 bytes, with a total purse of 500,000 euros paid out in proportion to the improvement [9]. The rules are the two-part code made concrete: the score is the size of the compressed file *plus* the size of the decompressor, and the decompressor must run under fixed limits of time and memory on a single processor core, without a graphics processor. Mahoney [17] maintains a companion benchmark without resource limits. Figure 10 shows where the general-purpose compressors and the specialists stand.

Three things in the figure are worth pointing out to students. The gap from `gzip` to `xz` is the gain from longer contexts and better entropy coding within the same dictionary idea; the gap from `xz` to the record is the gain from *modelling*, that is, from putting a better sequential probability assignment in front of the arithmetic coder, which is the only thing the record holders do differently. The record has moved by a few percent a year for two decades. And the best systems are already below Shannon's human number, on a task that is not quite Shannon's task.

Language models as compressors. The record holders of Figure 10 are compressors built to compress. Language models are trained by minimising (31) on text and are then used to write; Theorem 7.2 says they are compressors whether or not anyone uses them as such, and Delétang et al. [4] used them as such. Driving an arithmetic coder with the conditional probabilities of a 70-billion-parameter model trained only on text, they compressed patches of the ImageNet photographs to 43.4% of their size, against 58.5% for PNG, and speech from LibriSpeech to 16.4%, against 30.3% for FLAC: a text model, never shown a pixel or a sound sample, is a

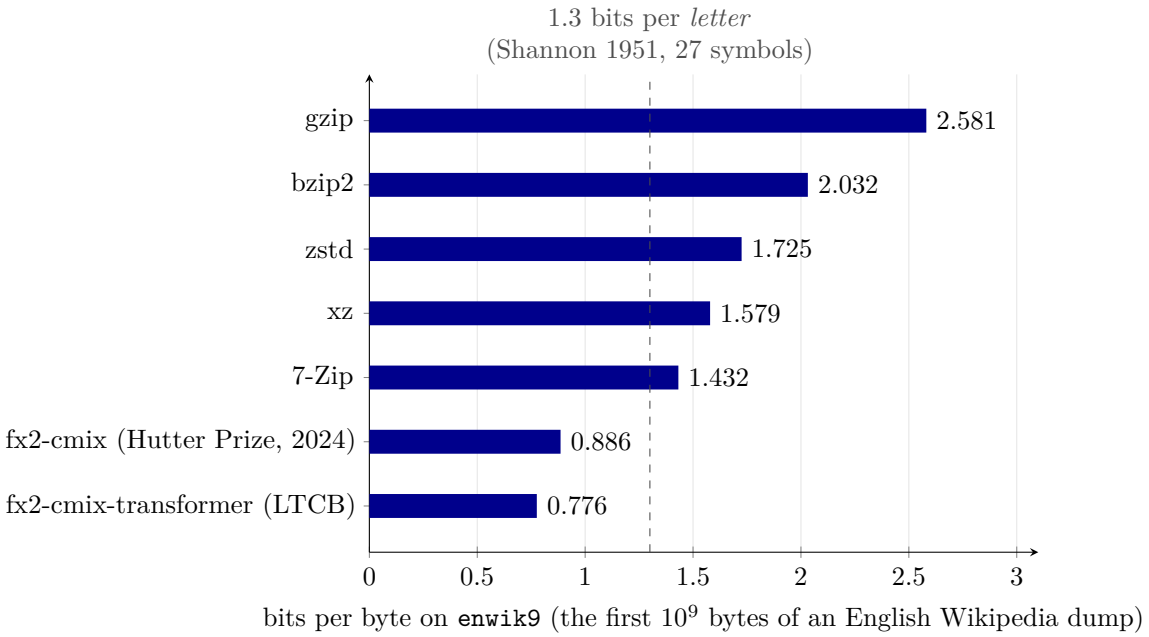


Figure 10: Compressed size of `enwik9` in bits per byte. The general-purpose tools descend from Ziv and Lempel's dictionary methods [25] with adaptive arithmetic or Huffman coding of the output; the two record holders are context-mixing compressors, ensembles of hundreds of adaptive models whose predictions are combined by small neural networks and fed to an arithmetic coder, exactly the architecture of Section 6.2. The dashed line is Shannon's per-letter estimate for human readers on a 27-symbol alphabet; `enwik9` is bytes of Wikipedia markup, so the line is a landmark, not a fair opponent. Sizes from Mahoney [17] and Hutter [9], converted in `checks/verify_examples.py`.

better image and audio compressor than the formats built for the purpose, because in-context prediction of any sequence is what it was trained to do. The same paper shows the converse of Theorem 7.2(i) at work: since a compressor defines a model q_C, one can *sample* from `gzip`, and the samples look like the statistics of the file it was fed. And it keeps the two-part bill: counted with the size of its parameters, some 140 gigabytes for the 70-billion-parameter model, a model that large is a hopeless compressor of a one-gigabyte file, and in the paper's own two-part accounting the winners at that scale are Transformers with millions, not billions, of parameters.

Huang et al. [7] tested the thesis in its strongest form. Taking some thirty publicly available language models of different sizes and families, they measured the bits per character each achieves on fresh corpora of three kinds, general knowledge, program code and mathematics, and compared them with the models' average scores on the standard benchmarks for the corresponding abilities. The relation is close to linear across models, with correlation coefficients near -0.95 in each domain: the fewer bits per character a model needs for a kind of text, the better it does on tests of the ability that kind of text exercises. Compression, on this evidence, is at least a very good proxy for what the benchmarks measure.

Scaling laws and the irreducible loss. Finally, the training loss of language models behaves with a regularity that information theory predicts in shape if not in exponent. Kaplan et al. [11] found that the loss of Transformer models [23] falls as a power law in the number of parameters N, the number of training tokens D and the compute spent, with small exponents, over more than seven orders of magnitude of compute. Hoffmann et al. [6] fitted the form

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \qquad E = 1.69,\ A = 406.4,\ B = 410.7,\ \alpha = 0.34,\ \beta = 0.28, \quad (33)$$

in nats per token, to a large family of models, and concluded that compute is best spent with about 20 training tokens per parameter. Figure 11 plots (33) along that rule in bits per token. The two variable terms are the cost of a finite model and of finite data, the two ways of not knowing the source; the constant $E = 1.69$ nats $= 2.44$ bits per token is what remains when both go to zero, and by Theorem 7.2(iii) it is the fit's estimate of the *entropy rate* of the token stream, the quantity Shannon estimated for letters at 1.3 bits with human predictors. The estimate is only as good as the parametric form and is an extrapolation, but its meaning is unambiguous, and it is a fine question for students whether it can be checked by other means.

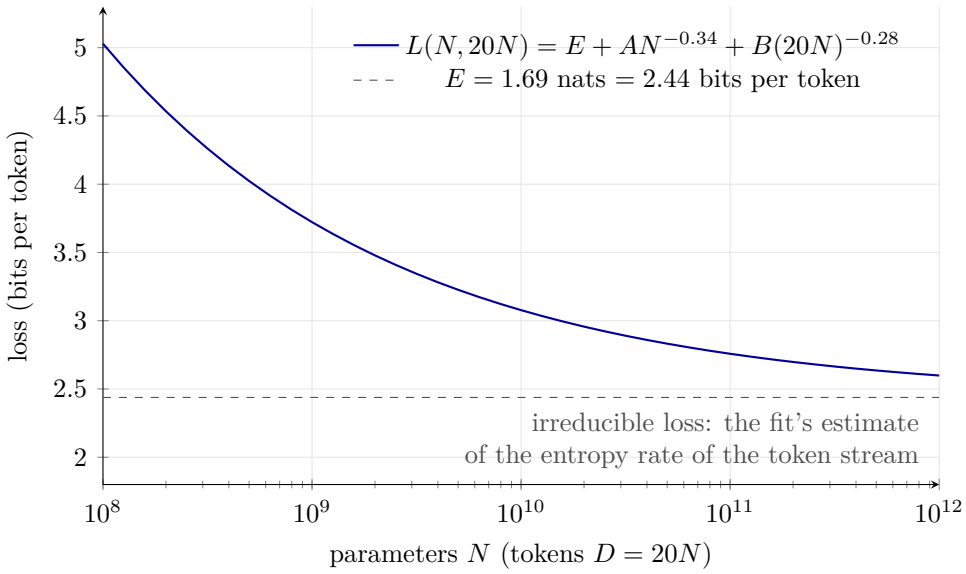


Figure 11: The parametric fit (33) of Hoffmann et al. [6], evaluated along $D = 20N$ and converted to bits per token. The curve is the published formula, not measured points; the fit was made on models between roughly 7×10^7 and 1.6×10^{10} parameters, so the ends are extrapolations. The asymptote E is the fit's estimate of the entropy rate of the token stream.

7.4 What the thesis does and does not say

The evidence of Section 7.3 is real and the theorems behind it are exact, and it is precisely because students will find the thesis attractive that its limits should be stated with the same care. Six of them follow; each is a place where a computational project can go wrong.

Compression is relative to data. A model compresses a corpus, and the bits it saves measure how well it predicts *that* distribution. The text model of Delétang et al. [4] compresses images somewhat, not because it understands photographs but because it predicts sequences in context; a claim about "intelligence" is always a claim about a class of environments, which is why Legg and Hutter [15] average over one.

Held-out data or nothing. On its training data a model's loss measures memorisation, not prediction: the gambler is betting on races whose results he has read. All the numbers of Section 7.3 are on data the systems had not seen, or are two-part scores that charge for the model, and the modern worry about benchmark contamination is this point restated. In Section 7.5 the held-out text is fixed before anything is trained and is never used to choose anything.

The bill includes the model. A loss per symbol ignores the description length of the model, which the two-part code and the Hutter Prize count. At 10^9 bytes of data a 10^{11}-byte model is a bad compressor whatever its loss; the thesis concerns the regime where the data are much larger than the model, or the online regime where the model is learned from the data being compressed. A project should report which regime it is in.

Units are not optional. Bits per token depend on the tokeniser; a larger vocabulary gives a larger loss per token and says nothing by itself about the loss per character. Remark 7.3 is the only defence.

Induction is not all of intelligence. Solomonoff's M is the optimal *predictor*; Hutter's agent adds a planning half that chooses actions, and the definition of Legg and Hutter [15] is about achieving goals, not about describing observations. Compression measures the perception and induction half. Whether the other half comes for free with it is an open question that the linear correlations of Huang et al. [7] bear on but do not settle: they were measured across models built by people, on benchmarks built by people, and a correlation across such a population is not a mechanism.

Below the entropy there is nothing. Theorem 7.2(iii) puts a floor under every model. When a learning curve flattens it may be the model's limit or the source's entropy rate, and only an independent estimate of the rate, as in (33) or by Shannon's and Cover and King's methods, tells the two apart. The starter experiment of Section 7.5 is built on a source whose rate is known exactly, for this reason.

7.5 The final activity suggestion: reading a small language model as a compressor

Students who choose the computational challenge will train a small language model on a text corpus of their choice, with whatever model and software they prefer, and then *measure it as a compressor* against the baselines of this course. The choice of network, library and hardware is theirs; the measurement is not. What follows is the measurement contract, a starter experiment on which every number is known in advance, and what the report should contain.

> **Convention 7.5** *(Measurement contract).* Every model, baseline and compressor in the project is scored by the same quantity: bits per character (or per byte, stated once) on one *held-out* text, fixed before any training or tuning starts and never used to choose anything. Losses in nats are divided by $\ln 2$; losses per token are multiplied by tokens per character of the held-out text; compressors are scored by $8 \times$ (compressed bytes)/(characters). The alphabet and its size m are stated, and $\log m$ is the first row of every table.

Corpus. Two kinds of corpus fit the course. `text8` [17] is 10^8 characters of English Wikipedia reduced to the 27 symbols `a`–`z` and space, that is, Shannon's alphabet of Section 6.3, so its numbers can be set beside Section 6.4 directly; the first million characters are plenty for a small model and a laptop. Alternatively, a corpus of Portuguese prose in the public domain (the novels of Machado de Assis, for instance) reduced to a fixed alphabet of letters, accented letters, space and a few punctuation marks raises a question the English literature does not answer, namely what the entropy rate of Portuguese is, and gives the project an original result. Either way, hold out the final ten percent as the test text of Convention 7.5, keep a further slice for validation during training, and train on the rest.

Baselines. The baselines are the course itself, and they must be computed on the held-out text before the model is. The uniform model, $\log m$ bits per character. The order-0 model, letter frequencies estimated on the training text, whose loss on the test text should land near Shannon's 4.03 for English [21]. The n-gram models of orders 1 to 5, conditional frequencies with add-$\frac{1}{2}$ smoothing (the $\alpha = \frac{1}{2}$ of the Krichevsky–Trofimov estimator; any standard smoothing will do, stated), which reproduce Shannon's table of F_n and show the order at which sparse counts start to hurt. The general-purpose compressors `gzip -9`, `bzip2`, `xz -9` and `zstd -19` on the test text alone, scored per Convention 7.5; they are adaptive, so they need no training text, and a fair variant that lets them use it is to compress the concatenation of training and test text and subtract the size of the compressed training text. An n-gram model beating `xz`

on a million characters is the first thing worth seeing; `xz` beating the order-5 model on a long text is the second.

The model. Any sequential model with a softmax over the alphabet will do; a character-level Transformer of a few hundred thousand to a few million parameters trains in minutes to hours on a laptop or a free cloud notebook, and a compact reference implementation such as `nanoGPT` [12] exists for those who want a starting point rather than a blank file. Work at the character or byte level unless there is a reason not to; tokenisers add a conversion and remove nothing from the science. Record the validation loss in bits per character as a function of the number of training characters seen, and train at least three model sizes to the same number of characters so that a plot of loss against N on logarithmic axes can be compared, in shape, with Figure 11. Record the training loss too: the gap between training and validation loss is the memorisation of the second caveat, made visible.

The two-part bill. For each trained model, add the description length of the model to the compressed size of the test text: the number of parameters times the bits per parameter stored, or, better, the size of the compressed checkpoint, since the parameters are themselves a file that `xz` can shorten. Report the total against `xz` on the same test text, and estimate the length of test text at which the model would break even. This one number tells the reader whether the project is in the regime the thesis speaks of.

Optional: make it a real compressor. The most convincing report closes the loop of Figure 8: implement an integer arithmetic coder along the lines of Witten et al. [24], drive it with the model's probabilities quantised to integers, decode the output with an identical copy of the model, and check that the decoded text is byte-for-byte the original. The actual file size should exceed $\sum_t \log 1/q(x_t \mid x^{t-1})$ by a handful of bits; any larger discrepancy is a bug, and any failure to decode is the twin problem in person, most often floating-point nondeterminism, which is why the probabilities are quantised before use.

Example 7.6 *(Starter experiment: a source whose entropy rate is known).* Before touching real text, run the whole pipeline on the order-1 Markov chain on four letters defined in `checks/verify_examples.py` (block "Lecture 7"). Its entropy rate is $H(\mathcal{X}) = 1.3984$ bits per symbol and its one-letter entropy is $H(X_1) = 1.9743$, against $\log 4 = 2$. On a held-out million symbols generated by the script, the baselines come out as follows: the uniform model costs 2.0000; the order-0 model 1.9743, which is $H(X_1)$ to four decimals; the order-1, 2 and 3 models 1.3995, 1.3995 and 1.3996, which is the entropy rate plus the small cost of estimating the transition counts, with no gain from orders the source does not have; and the compressors `gzip` (through `zlib`), `bzip2` and `xz` cost 1.7361, 1.7205 and 1.6117, all well above the rate, because their dictionary methods look for long repeated strings and a short-memory Markov source offers few. A neural model trained on the first million symbols should approach 1.3984 from above, with the same small statistical excess as the n-gram models; a value clearly below the rate means the test text has leaked into training. The excess of the uniform model over the rate, $2 - 1.3984 = 0.6016$ bits, is $D(p \,\|\, \text{uniform})$ per symbol, the wrong-code theorem in numbers.

References

[1] Fabrice Bellard. NNCP v2: Lossless data compression with Transformer. Technical report, https://bellard.org/nncp/, 2021. A compressor that trains a Transformer online while

coding, the decoder retraining an identical copy: the online solution of the twin problem (Session 7).

[2] Thomas M. Cover and Roger C. King. A convergent gambling estimate of the entropy of English. *IEEE Transactions on Information Theory*, 24(4):413–421, 1978. The gambling estimate $\hat{H} = \log 27 - \frac{1}{n} \log S_n$ and the value 1.34 bits per letter (Session 6).

[3] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2nd edition, 2006. The course textbook. Sessions 1–4 follow Chapter 5 and Sessions 5–6 follow Chapter 6; theorem numbers quoted in the collaborator notes refer to this edition.

[4] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.10668. Large language models driving an arithmetic coder compress text, images and audio; the two-part ("adjusted") accounting that charges for the model (Session 7).

[5] Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, Cambridge, MA, 2007. Modern reference on MDL, including prequential (online) coding (Session 7).

[6] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. arXiv:2203.15556. The parametric fit $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ plotted in Session 7; its constant E is read there as an estimate of the entropy rate.

[7] Yuzhen Huang, Jinghan Zhang, Zifei Shan, and Junxian He. Compression represents intelligence linearly. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2404.09937. Bits per character on fresh corpora against benchmark scores across many language models: a near-linear relation (Session 7).

[8] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952. The algorithm and optimality argument of Session 3.

[9] Marcus Hutter. The Hutter Prize for lossless compression of human knowledge. `http://prize.hutter1.net`. Rules and record table for `enwik8` and `enwik9`; the record size quoted in Session 7 was read in September 2026 and will change.

[10] Marcus Hutter. *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer, Berlin, 2005. Solomonoff induction as the foundation of a theory of general agents; the intellectual basis of the Hutter Prize (Session 7).

[11] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. arXiv:2001.08361. Power-law dependence of the loss on parameters, data and compute (Session 7).

[12] Andrej Karpathy. nanoGPT. Software repository, `https://github.com/karpathy/nanoGPT`, 2022. A compact character- or token-level Transformer training script, mentioned

in Session 7 as one optional starting point among many; the final activity prescribes no framework.

[13] John L. Kelly, Jr. A new interpretation of information rate. *Bell System Technical Journal*, 35(4):917–926, 1956. Proportional betting, the doubling rate and the identification of the gambler's gain from side information with the channel's information rate (Session 5).

[14] Andrei N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965. Kolmogorov complexity (Session 7).

[15] Shane Legg and Marcus Hutter. Universal intelligence: A definition of machine intelligence. *Minds and Machines*, 17(4):391–444, 2007. A formal definition of intelligence as goal achievement across environments weighted by complexity; used in Session 7 to say what compression measures and what it leaves out.

[16] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, Cham, 4th edition, 2019. Reference for the facts about K and for the dominance bound of Solomonoff's predictor quoted in Session 7.

[17] Matthew Mahoney. Large Text Compression Benchmark. `http://mattmahoney.net/dc/text.html`; the `text8` and `enwik9` data are described at `http://mattmahoney.net/dc/textdata.html`. Compressed sizes of `enwik9` for the general-purpose compressors and the unrestricted entries of the figure in Session 7 (read September 2026); source of the `text8` corpus recommended for the final activity.

[18] Matthew V. Mahoney. Text compression as a test for artificial intelligence. In *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI-99)*, page 970. AAAI Press, 1999. Proposes the compressed size of text as an objective test of language understanding (Session 7).

[19] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978. The minimum description length principle: two-part codes as model selection (Session 7).

[20] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423 and 623–656, 1948. Source coding theorem, the entropy bound of Session 2, the Shannon code, and the stochastic models of English sampled in Session 6.

[21] Claude E. Shannon. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1):50–64, 1951. The guessing game and the estimates $F_0 = 4.76$, $F_1 = 4.03$, $F_4 \approx 2.8$ and about 1.3 bits per letter for human predictors (Session 6); the human landmark in the figure of Session 7.

[22] Ray J. Solomonoff. A formal theory of inductive inference, Parts I and II. *Information and Control*, 7:1–22 and 224–254, 1964. The universal prior and universal sequential prediction (Session 7).

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008, 2017. The Transformer architecture used by every neural system mentioned in Session 7; cited for the architecture only.

[24] Ian H. Witten, Radford M. Neal, and John G. Cleary. Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–540, 1987. The standard exposition, with working integer code; the adaptive coder is the online twin of Session 7, and the recommended reference for students who implement the optional coder of the final activity.

[25] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977. The dictionary method behind `gzip`, `xz` and `zstd`, the general-purpose baselines of Session 7.